

Runtime Code Generation for Constant-Time Secret-Indexed Array Accesses: Applications to PERK and NTRU

Décio Luiz Gazzoni Filho¹, Rafael G. Flores e Silva¹, Alessandro Budroni²,
Marco Palumbi², and Gora Adj²

¹ Dept. of Electrical Engineering, State University of Londrina, Londrina, Brazil,
{dgazzoni,rafael.guilhermefs}@uel.br

² Technology Innovation Institute, Abu Dhabi, UAE,
{alessandro.budroni,marco.palumbi,gora.adj}@tii.ae

Abstract. One of the main guidelines to prevent timing side-channel attacks against cryptographic implementations is to avoid array accesses indexed by secret data. However, alternatives and countermeasures often incur significant performance losses. We propose a novel methodology for secure, constant-time implementation of algorithms that read and write to small arrays with secret-dependent indices, with a constant-factor performance impact compared to timing-unprotected accesses. It is specifically suitable for simple in-order CPUs like those in embedded systems, e.g., the ARM Cortex-M4 core. Although our methodology is general, we illustrate it with secure implementation of permutation operations, such as composition, inversion, and sampling, the latter using the Fisher-Yates shuffle. We apply this methodology to the post-quantum cryptosystems PERK and NTRU, bridging most of the performance gap to unprotected implementations that employ secret-dependent array accesses.

Keywords: Shuffling · Constant-time implementation · PERK · NTRU
· Post-quantum Cryptography

1 Introduction

Side-channel attacks are a class of implementation-level attacks that exploit physical information unintentionally leaked by a computing device during cryptographic operations. Such information includes timing variations, power consumption traces, electromagnetic emissions, or cache access patterns. These leakages can be measured and analyzed to extract secret keys or data, without directly solving the underlying hardness assumption of the scheme implemented.

To prevent such attacks, secure cryptographic implementations must follow strict development guidelines. For example, algorithms should always run in constant time with respect to secret inputs, and when accessing arrays, indices must be independent of secret data. Unfortunately, adhering to these principles often comes at a significant performance cost increase. For instance, some of the

fastest implementations of the Advanced Encryption Standard (AES) use pre-computed lookup tables to speed up substitution and mixing operations. However, such implementations typically involve memory accesses indexed by secret key or plaintext bytes, which can leak sensitive information through cache-based side-channel attacks [16]. To mitigate this, constant-time variants replace table lookups with arithmetic or bit-sliced operations that eliminate secret-dependent memory access [4, 48]. While these approaches improve security, they come at a notable performance cost, especially on general-purpose CPUs where table lookups are efficient and cache access is fast. This trade-off illustrates the broader challenge of balancing side-channel resistance with computational efficiency in cryptographic implementations. Another relevant example is the Fisher–Yates shuffle [28], a standard algorithm for sampling permutations and fixed Hamming-weight vectors uniformly at random. Its time complexity is $O(n)$ in its basic form. However, when modified to avoid secret-dependent memory accesses for side-channel resistance, its complexity increases to $O(n^2)$ [21, 49]. Other alternatives to perform such operations involve constant-time sorting algorithms based on sorting networks [17, 18], which exhibit a time complexity of $O(n \log^2 n)$, and an $O(n)$ constant-time algorithm specific to fixed weight sampling [27].

Our contribution. This work presents a novel approach specifically designed to eliminate secret-dependent memory accesses. The core idea is to leverage the CPU’s register file as temporary storage for arrays. Unlike memory, registers are not subject to caching effects³ and thus do not leak information through access patterns based on secret indices. While registers cannot be indexed dynamically in machine code — since register operands are hardcoded in instruction encodings — we overcome this limitation by generating code at runtime that encodes the desired register accesses directly into the instruction stream. This enables the secure implementation of algorithms such as the Fisher–Yates shuffle without secret-dependent memory operations. We demonstrate that our approach outperforms constant-time algorithms for permutation and fixed-Hamming weight vector sampling based on sorting, already accounting for the cost of both code generation and execution of the generated code. For instance, our method achieves a $5\times$ speed-up for permuting vectors and an $8\times$ speed-up for inverting permutations, compared to constant-time sorting-based techniques. Furthermore, we evaluate the overall impact of our technique within the post-quantum schemes PERK [1] and NTRU [22] on an embedded platform based on the ARM Cortex-M4 core, observing overall consistent performance improvements.

Related work. The problem of protecting cryptographic implementations on cache-enabled architectures from timing attacks has been extensively studied

³ While registers may be written back to memory during e.g. context switches or interrupts – and thus pass through the cache memory subsystem – such accesses are performed in a data-independent fashion and, to the best of our knowledge, do not introduce new avenues for timing side-channel attacks.

in the context of AES. One early and influential mitigation is the use of vector permute instructions, introduced by Hamburg [29]. Another widely adopted technique is *bitslicing*, originally proposed by Biham for DES [20], and later adapted to AES in several implementations [33, 37, 40, 41, 48]. More recently, Adomnicai and Peyrin presented an efficient bitsliced AES implementation tailored for embedded platforms [4].

In the context of sampling fixed-Hamming weight vectors and permutations, Bernstein et al. [18] proposed the use of constant-time sorting networks as a general countermeasure. Alternative algorithms for sampling fixed-Hamming weight vectors were proposed by Sendrier [49] and by Gazzoni Filho et al. [27]; the former, while asymptotically slower, achieves acceptable performance in the sparse vector setting, whereas the latter exploits the fact that the vector has a small, fixed set of entries to obtain a constant-time $O(n)$ variant of the Fisher-Yates shuffle. A comprehensive systematization of algorithms for sampling random permutations was recently made by Budroni et al. [21].

Artifacts. The source code relative used to run the experiments in this paper is published under an open-source license.⁴⁵

Organization. We give in Section 2 the necessary notation and background notions. In Section 3 we introduce our approach for runtime code generation for constant time array accesses indexed by secret data. We display secure algorithms for permutation operations, and how they apply to the PERK and NTRU schemes, in Section 4. We collect and analyze performance results for these schemes in Section 5. Finally, we give our conclusions and future directions in Section 6.

2 Preliminaries

Notation. We denote the set of natural, integer and real numbers as $\mathbb{N}$, $\mathbb{Z}$ and $\mathbb{R}$, respectively. We use calligraphic capital letters (e.g., $\mathcal{A}$) to denote sets. The notation $U(\mathcal{A})$ denotes the uniform distribution over a finite set $\mathcal{A}$. The notation $r \leftarrow U(\mathcal{A})$ indicates that the element r has been sampled uniformly at random from the set $\mathcal{A}$. The notation $|\cdot|$ is used to indicate either the cardinality of a set (e.g., $|\mathcal{A}|$) or the absolute value of a real number (e.g., $|a|$, for $a \in \mathbb{R}$). We denote with $[m, n]$ the set $\{m, m + 1, \dots, n - 1\}$, for integers $m \leq n$, and $[n] = [0, n] = \{0, 1, \dots, n - 1\}$.

2.1 Timing side-channel attacks

Timing side-channel attacks leverage variations in execution time that depend on secret data to recover sensitive information. This concept was introduced in

⁴ <https://github.com/Crypto-TII/arm-shuffling-codegen>.

⁵ <https://github.com/Crypto-TII/arm-shuffling-codegen-pqm4>.

the seminal work of Kocher [36] in 1996, demonstrating that cryptosystems like Diffie-Hellman and RSA could leak secrets through variations in the execution time of certain modular exponentiation algorithms; it was first demonstrated in practice by Dhem et al. [25] by successfully extracting the private key from a smart card implementation of the RSA cryptosystem.

Tsunoo et al. [50, 51] observed that cache memory, present in many modern CPUs, could give rise to variable timings for table lookups. This was published soon after the standardization of the AES block cipher [5], which heavily relied on table lookups, with indices correlated with the plaintext or secret key, for efficient software implementation in CPUs of the time. Bernstein [16] demonstrated how to mount a cache timing attack on AES, assuming that a remote attacker is able to trigger AES operations and precisely measure their execution timings. The attacker can recover key-dependent information by correlating timing variations with known plaintext inputs. This cemented the understanding that secret-dependent array accesses are potentially a devastating vulnerability in cryptographic implementations.

Cache attacks were only the beginning, as many other attacks targeted microarchitectural leaks in modern, large (PC- or mobile-class) CPUs; a few examples are Spectre [35], Meltdown [39] and Hertzbleed [52]. It is now widely recognized that cryptosystems must be carefully implemented on such CPUs to avoid these attacks. However, CPU cores found in embedded systems, e.g. those based on the ARM M-profile architecture, employ much simpler microarchitectures, and the literature on timing side-channel attacks targeting embedded systems is not as rich. However, as embedded microarchitectures evolve, it seems inevitable that they may be subject to microarchitectural leakage attacks, whether previously-known types or new attacks that exploit particularities of embedded systems.

For instance, cache memories were historically less prevalent in embedded systems. However, the increasing clock speed of embedded cores, coupled with physical limitations on the access times of Flash memory (currently the most prominent memory technology for non-volatile storage), led to the introduction of Flash memory caches, e.g., the ART Accelerator found in ST Microelectronics parts [42, Section 3.3.4]. Thus, tables stored in more plentiful non-volatile storage become vulnerable to cache timing attacks. On the other hand, static RAM (SRAM), used for volatile memory, can keep up with core clock speed increases and is usually uncached⁶. Even so, other mechanisms are conjectured to induce timing variabilities in SRAM accesses (see [15, p. 14] and [45, p. 8]); recently, side-channel attacks have been demonstrated on ARM M-profile CPUs by exploiting a vulnerability of this kind, i.e., timing differences in the memory bus arbitration logic [46]. In response to this attack, ARM issued guidance [13] reinforcing the need to ensure that “control flow and memory access patterns created by the program do not depend on a secret state”.

⁶ However, Matthias Kannwischer (personal communication) reports that, on a Cortex-M85 part (Renesas RA8M1), enabling RAM data cache results in noticeable performance improvements.

Threat model. The goal of our proposed technique is to protect implementations against timing side-channel attacks. To that end, we seek to eliminate any execution time variations that are correlated with secret data; this is usually referred to, somewhat imprecisely, in the implementation literature as *constant-time execution*, a terminology which we employ regardless, given its prevalence. In particular, our technique is concerned with avoiding timing variations due to array accesses indexed by secret data, which can be eliminated from implementations (through our proposed technique or other countermeasures) by replacing such accesses with other kinds of operations that are widely assumed to be constant-time. For instance, Pornin [44, Section “Generic Tools/Execution Model”] claims that “(m)ost elementary operations (opcodes in the machine language) are constant-time”, and proceeds to list potentially non-constant-time operations: memory accesses (indexed by secret data), conditional jumps, integer divisions, shifts and rotations (with secret shift/rotation amounts) and multiplications. This mirrors the prevailing view of the cryptographic implementation community. When arguing that some of our implementations are constant-time, we only discuss issues related to such potentially non-constant-time operations.

2.2 Permutations

Let n be a positive integer. The set of all bijections $[n] \rightarrow [n]$ is denoted by $\mathcal{S}_n$ and is referred to as the *symmetric group of order n* . We call a permutation an element π of $\mathcal{S}_n$ and represent it as

$$\pi = \begin{pmatrix} 0 & 1 & \dots & n-1 \\ \pi(0) & \pi(1) & \dots & \pi(n-1) \end{pmatrix} =: (\pi(0), \pi(1), \dots, \pi(n-1)),$$

meaning that π acts on the elements of $[n]$ by sending i to $\pi(i) = j \in [n]$, for some $0 \leq j < n$. $\mathcal{S}_n$, paired with the composition operation $\circ$, forms a group. The identity element is the identity permutation $id = (0, 1, \dots, n-1) \in \mathcal{S}_n$ and $|\mathcal{S}_n| = n!$. Also, we denote with π^{-1} the inverse π , i.e., $\pi \circ \pi^{-1} = id$. As in PERK [1], we adopt the convention that the resulting vector o of applying π to a size- n vector v is computed as $o[i] = v[\pi^{-1}(i)]$ for $i \in [n]$.

In cryptography, it is often required to sample random permutations from the set $\mathcal{S}_n$. Generally, there are two main strategies used in practice: *Fisher-Yates shuffle* and *sorting*. We give below a brief review of these two approaches. For more details, we refer the reader to [21].

Fisher-Yates shuffle. To sample a permutation uniformly at random from the symmetric group $\mathcal{S}_n$, one can sequentially select elements from the set $[n]$ without replacement and record them in the order selected. Sampling without replacement is computationally non-trivial, but can be efficiently achieved using the Fisher-Yates shuffle algorithm [28], which we report in its modern version due to Durstenfeld [26] in Algorithm 1.

Assuming that the *swap* operation has complexity $O(1)$, the Fisher-Yates shuffle has both time and memory complexity $O(n)$, making it the most efficient known algorithm for uniform random sampling of permutations. However,

Algorithm 1 Fisher-Yates shuffle.**Input:** A positive integer n **Output:** A random permutation $\pi \in \mathcal{S}_n$

-
- ```

1: Initialize $\pi = (0, 1, \dots, n-1)$
2: for $i = n$ down to 1 do
3: $j \leftarrow U([i])$
4: $\text{swap}(\pi(i), \pi(j))$
5: return π

```
- 

when used in a cryptographic context, it presents a critical flaw: if the permutation must remain secret, the sampling routine must be resistant to side-channel attacks. The *swap* operation performs memory accesses that depend on secret indices of the permutation, which, as exposed in Section 2.1, is a practice generally discouraged in secure cryptographic implementations. To mitigate this issue, side-channel-resistant variants of the Fisher–Yates shuffle have been proposed [21, 49], albeit with increased time complexity of  $O(n^2)$  when sampling a full size- $n$  permutation.

**Sorting.** Another approach used in cryptography to sample permutations is based on sorting algorithms. Consider the following list of pairs

$$((x_0, 0) (x_1, 1) \cdots (x_{n-1}, n-1)), \quad \text{for } x_0, \dots, x_{n-1} \leftarrow U([M]),$$

for an integer  $M \geq n$ . If we sort this list with respect to the  $x_i$  values on the left-hand side of each entry, the values on the right-hand side would get *permuted*. This routine allows to sample a random permutation, reported in Algorithm 2, whose time complexity depends on the sorting subroutine. One important consideration to make here is that there must not be any repetition in the list of integers  $x_i$  to sort in order to avoid the introduction of any bias.

In cryptography, the main sorting algorithm used [17, 18] is based on sorting networks, with a time complexity of  $O(n \log^2 n)$  [34, Sec. 5.3.4]; it is employed in post-quantum schemes such as McEliece [6], NTRU [22], and PERK [1].

**Permutation composition and inversion.** Besides sampling, it is often required to implement additional operations on permutations, such as inversion, composition and application to a vector. The naïve algorithms for these operations are as follows.

- **Composition.** Let  $\pi_1, \pi_2 \in \mathcal{S}_n$ . One computes the composition  $\tau := \pi_1 \circ \pi_2$  by setting  $\tau(i) = \pi_1(\pi_2(i))$ , for  $i \in [n]$ .
- **Inversion.** Let  $\pi \in \mathcal{S}_n$ . One computes its inverse  $\pi^{-1}$  by setting  $\pi^{-1}(\pi(i)) = i$ , for  $i \in [n]$ .
- **Application to a vector.** Let  $\mathbf{v} \in \mathbb{F}^n$  be a vector with coefficients in a field  $\mathbb{F}$ , and  $\pi \in \mathcal{S}_n$  be a permutation. One computes  $\mathbf{w} \in \mathbb{F}^n$ , the permutation of  $\mathbf{v}$  by  $\pi$ , by setting  $\mathbf{w}(i) = \mathbf{v}(\pi(i))$ .

**Algorithm 2** Sorting-based random permutation sampling.**Input:** Positive integers  $n, M$  such that  $n \leq M$ , a sorting algorithm  $sort(\cdot)$ **Output:** A random permutation  $\pi \in \mathcal{S}_n$ 


---

```

1: Initialize a list ℓ of length n
2: repeat
3: for $i = 0$ to $n - 1$ do
4: $x \leftarrow U([M])$
5: $\ell(i) = (x, i)$
6: $\ell' = sort(\ell)$ # according to the left-hand side entries
7: Extract the lists τ, π from ℓ' such that $\ell'(i) = (\tau(i), \pi(i))$, $0 \leq i \leq n - 1$
8: until There are no repetitions in τ
9: return π

```

---

Similar to Fisher–Yates, the above algorithms have a time complexity of  $O(n)$  but present the same problem of secret-dependent memory accesses when the permutations are private. Constant-time alternatives for these algorithms, either using sorting or a similar approach as Sendrier’s variant of the Fisher–Yates shuffle, have been studied and compared in [21, Appendix A] and present an increase in the time complexity.

### 2.3 Runtime code generation

The first digital computers, arising during or shortly after World War II, used mechanisms such as plugboards for programming; thus, reprogramming them is laborious and time-consuming. Shortly afterwards, computers almost universally adopted the *stored program* concept, in which the computer’s instructions are stored in memory, just as its data. This implies that a computer program itself may manipulate instructions, a fact widely exploited by development tools such as compilers, assemblers, linkers, etc. Originally, such tools performed code generation *ahead of time*, at compilation or link time, but as processing power increased, *interpreted code* became popular for non-performance-critical applications. In programming environments such as Sun Microsystems’ Java and Microsoft’s .NET runtimes, source code is compiled to bytecode which runs in a virtual machine, allowing for portable binaries across different architectures. To bridge the performance gap between interpreted and compiled code, the Just-In-Time (JIT) paradigm was adopted; rather than executing code in an interpreter loop, with recurring performance penalties, JIT environments pay a one-time cost to translate bytecode into machine code<sup>7</sup>, achieving performance characteristics that are closer to compiled code, and can benefit from optimizations specific to the CPU running the code. Currently, JIT compilers are also widely employed for the Javascript programming language that powers the client side and, increasingly, the server side as well of the World Wide Web. This shows that the core idea behind this paper, of *runtime code generation*, has a rich history.

---

<sup>7</sup> Some environments perform another translation pass if a given block is identified as a hotspot, meriting extra optimization; this can still be viewed as a bounded cost.

Decades ago, when computers were much less powerful, but there was interest in computationally intensive applications such as graphical interfaces and video games, code generation and the related concept of *self-modifying code* (essentially, meshing the code generation and the execution of the generated function in a single pass) were fairly popular. Examples of its use in computer graphics were the runtime generation of specific operations in the fundamental *bit block transfer* or bit-blit operation, which applies bitwise operations to the frame buffer, loosely analogous to using bitslicing in cryptography. Graphics libraries for Microsoft Windows<sup>8</sup> and the Blit experimental bitmap terminal [43] achieved the required performance through on-the-fly code generation: given the desired bitwise operation to apply, they synthesized and then ran code to apply the specific operation to the selected area of the display.

A final example, which is quite close in spirit to our proposed technique, was the `int86` function provided by libraries of Intel PC development tools in the DOS era<sup>9</sup>. This function executes the Intel software interrupt instruction `INT`, which has an 8-bit parameter, the interrupt number. However, `int86` receives the interrupt number as an argument to the function call, i.e., it is a runtime, not compile-time, value. This would be easily implemented if the instruction could read the interrupt number from a register, but this is not the case [47, p. 281]: it is encoded as an 8-bit immediate field of the instruction<sup>10</sup>. To implement `int86`, a jump table (i.e., a `switch/case` statement) with all possible 256 interrupt numbers could be used, but this would take up a fairly large amount of memory for PCs of that period. An alternative that reduces code size is the use of self-modifying code [47, p 281], by patching into the encoding of the `INT` instruction to use the desired interrupt number, and then running the instruction.

## 2.4 The ARMv7-M architecture

In this section, we recall some details of the ARMv7-M architecture used by our target platform (the ARM Cortex-M4 core) and describe the main instructions used by our implementation.

The original ARM architecture is a 32-bit RISC load-store architecture. Its integer register file is 16 entries long, numbered from `R0` through `R15`. Of these, `R13` (`SP`, the stack pointer) and `R15` (`PC`, the program counter) are special-purpose and must not be directly used. Although `R14` is also reserved for use as the link register (`LR`), in practice, it can be repurposed by the programmer, so long as a copy is stored in the stack. ARM's original instruction encoding uses 32-bit words only, although an option was later introduced for a 16-bit encoding called Thumb, with a limited choice of instructions and accessible registers.

The seventh version of the architecture, released in 2003, was branched into three *profiles*: A (Application), R (Real-Time), and M (Microcontroller). The

<sup>8</sup> <https://devblogs.microsoft.com/oldnewthing/20180209-00/?p=97995>

<sup>9</sup> See e.g., the Borland Turbo C manual: <https://www.itlnet.net/programming/program/Reference/tc/ng2fc48.html>.

<sup>10</sup> <https://www.felixcloutier.com/x86/intn:into:int3:int1>



former is found in mobile phones, tablets, and laptops (and was extended to 64 bits with the release of the eighth version in 2011), but our focus in this work is on the M-profile, as implemented by various ARM Cortex-M cores. This architecture is known as ARMv7-M.

In particular, we are interested in the Cortex-M4 core [10], which conforms to ARMv7-EM, the ‘E’ indicating support for additional instructions referred to by ARM as “DSP extensions”. The Cortex-M4 optionally supports single-precision floating-point (FP) acceleration in hardware, in the form of additional instructions, and crucially, a dedicated FP register file with 32 entries, numbered from S0 through S31; the core may be denoted Cortex-M4F in this case. Although our technique does not perform any FP arithmetic, we do use the FP registers as additional storage for arrays<sup>11</sup>, and as such employ instructions to move data among memory, FP and integer registers. In practice, most commercially available Cortex-M4 parts include floating-point hardware.

Due to its application in the cost-sensitive embedded space and thus limited available memory, ARMv7-M employs the Thumb-2 instruction set, with a very similar programming model and choice of instructions; its main differentiating factor is the availability of both 16- and 32-bit instruction encodings, to improve code density. However, it is not a variable-length instruction set in the general sense, such as Intel’s x86, since only 16- and 32-bit encodings exist. Most instructions used in our technique, as demonstrated in this paper, only admit of 32-bit encodings, so for simplicity we declare arrays for generated code to have only 32-bit elements<sup>12</sup>.

Our implementations generate code for complete function calls rather than inserting instruction snippets in the middle of existing code. As such, the generated code must adhere to the ARM Architecture Procedure Call Standard (AAPCS32)<sup>13</sup>. We highlight the main points of interest for our implementations:

- If a function has at most 4 arguments, all of which are integers of width 32 bits or less or pointers, then such arguments are passed in the same order as found in the function prototype, through registers R0, R1, R2 and R3;
- Integer or pointer return values are stored in R0;
- The callee-saved integer registers are R4 through R12, and similarly for floating-point registers S16 through S31.

**Review of ARMv7-M instructions.** In Sections 3 and 4, we give many examples of algorithms employing our proposed technique (Algorithms 3 to 10). We thus review the operations performed by the instructions that we use in these algorithms, so that the reader unfamiliar with the ARMv7-M architecture may

<sup>11</sup> This technique has been previously used, e.g., in [2, 3, 7–9, 23, 24, 30] and others.

<sup>12</sup> Most instructions that can be encoded in 16 bits also have alternate 32-bit encodings, an important exception being the BX (branch and exchange) instruction, which is generally the last instruction in a code sequence; we pair it with a 16-bit NOP instruction, which does not actually execute as it follows the BX instruction.

<sup>13</sup> <https://github.com/ARM-software/abi-aa/blob/main/aapcs32/aapcs32.rst>

follow them. They are briefly described in Table 1; the relevant section in [12] is indicated, if a full description of a given instruction is sought.

| Instruction |                | Section<br>of [12] | Description                                                                             |
|-------------|----------------|--------------------|-----------------------------------------------------------------------------------------|
| BFI         | Rd, Rn, #l, #w | A.7.7.14           | $Rd[l + w - 1 : l] \leftarrow Rn[w - 1 : 0]$ .                                          |
| BX          | Rm             | A7.7.20            | Unconditional branch to address in Rm; to return from a function call, usually Rm = LR. |
| LDMIA       | Rn, {regs}     | A7.7.41            | Load 32-bit words from memory, starting at address Rn, to integer registers in {regs}.  |
| MOV         | Rd, Rm         | A7.7.77            | $Rd \leftarrow Rn$ .                                                                    |
| MOVW        | Rd, #i16       | A7.7.76            | $Rd \leftarrow i16$ .                                                                   |
| NOP         |                | A7.7.88            | No-operation.                                                                           |
| POP         | {regs}         | A7.7.99            | Pop integer registers in {regs} from the stack.                                         |
| PUSH        | {regs}         | A7.7.101           | Push integer registers in {regs} to the stack.                                          |
| STMIA       | Rn, {regs}     | A7.7.159           | Store integer registers in {regs} to 32-bit words in memory, starting at address Rn.    |
| STRB        | Rt, [Rn, Rm]   | A7.7.164           | Stores the least significant byte of Rt to a byte at the memory address Rn + Rm.        |
| UBFX        | Rd, Rn, #l, #w | A7.7.193           | $Rd[w - 1 : 0] \leftarrow Rn[l + w - 1 : l]$ .                                          |
| VLDmia      | Rn, {regs}     | A7.7.235           | Load 32-bit words from memory, starting at address Rn, to FP registers in {regs}.       |
| VMOV        | Sn, Rt         | A7.7.243           | $Sn \leftarrow Rt$ (bit-by-bit copy).                                                   |
| VMOV        | Rt, Sn         | A7.7.243           | $Rt \leftarrow Sn$ (bit-by-bit copy).                                                   |
| VPOP        | {regs}         | A7.7.251           | Pop the list of FP registers from the stack.                                            |
| VPUSH       | {regs}         | A7.7.252           | Push the list of FP registers to the stack.                                             |
| VSTMIA      | Rn, {regs}     | A7.7.258           | Store FP registers in {regs} to 32-bit words in memory, starting at address Rn.         |

**Table 1.** ARMv7-M instructions used in algorithms 3 to 10.

We briefly review the syntax of instructions displayed in the table. The first field corresponds to the instruction mnemonic. Fields starting with the letter R are integer registers, and the first such field after the instruction mnemonic is usually the destination register (most of the time, denoted by Rd), if there is one. Other (usually source) integer registers are referred to as Rm, Rn, and Rt. Floating-point registers start with the letter S, rather than R. Immediate fields, directly encoded into instructions, are preceded by the # character.

Some instructions (load/store multiple, push and pop) take a list of registers as an argument between braces, denoted in Table 1 as {regs}; usually, integer registers are encoded as bitfields in the instruction and thus may include an arbitrary set of such registers, whereas corresponding floating-point instructions encode only the first and last register indices of the range, and thus must be

consecutive. In our algorithms, only consecutive ranges were used, which we compactly represent using the same notation as ARM assembly language, e.g.,  $\{R0-R12\}$  represents the list of integer registers  $R0, R1, \dots, R12$ .

We adopt a pseudocode notation to simplify the description of some instructions in Table 1. Additionally, “slices” of bit registers are represented by  $[b_h : b_l]$ , to indicate that only bits  $b_h, b_{h-1}, \dots, b_{l+1}, b_l$  (numbered from most to least significant) are to be read or written.

Lastly, note that floating-point data movement instructions (between registers, either integer or floating-point, and/or memory) are done in a bit-by-bit fashion, oblivious to issues of representation as IEEE-754 floating-point numbers; therefore, no operations such as floating-point normalization are performed which might change the bit representation of operands.

### 3 Runtime Code Generation for Secure Array Accesses

As discussed in Section 2.1, array accesses indexed by secret data are vulnerable to cache timing attacks, which have been demonstrated in practice on larger out-of-order CPUs such as those found in PCs, laptops, servers, mobile phones, and tablets. Embedded systems are equally vulnerable in some cases, such as lookups from constant tables stored in cached Flash memory, and possibly even for RAM accesses.

One level of the memory hierarchy in computer systems that has been overlooked in previous works is the CPU register file(s). Registers almost always have zero-latency access times (i.e., their access time is included in instruction execution). As a corollary, access time is independent of which exact register is accessed by a given instruction. Our work investigates whether this property can be exploited to obtain constant-time array accesses indexed by secret data.

This idea may not have been considered previously due to the roadblocks of accessing CPU registers in an indexed manner. Recall from Section 2.4 that register indices are hardcoded into the instruction encoding in the ARMv7-M architecture, which is typical of most CPU architectures. A notable exception is SIMD extensions found in larger CPUs, some of which provide vector permutation instructions such as Intel x86-64’s `PSHUFQ` and ARM A-profile `TBL`, which may be repurposed to perform array lookups, a fact which Hamburg exploited to accelerate the AES block cipher [29]. However, such instructions are not available in embedded CPUs, even in ARM’s recently introduced Helium M-profile Vector Extensions (MVE) found in cores such as the Cortex-M55 and Cortex-M85; moreover, while Hamburg’s technique addresses the issue of *reading* from arrays, to the best of our knowledge no corresponding techniques for *writing* to arrays have been proposed, which is required in many algorithms such as those for permutation operations, displayed in Section 2.2.

**Our proposed approach.** We propose a new approach to utilize the CPU register file(s) to perform constant-time reads and writes to arrays with secret indices. Since indices are hardcoded in the instruction encoding, we overcome the

apparent impossibility of indexing the register file by *generating code at runtime* to perform such array accesses; i.e., secret data is directly encoded into instructions as register indices. We split the implementation of algorithms requiring protected array accesses into two passes: in the first, we run a code generator which writes executable (machine) code to RAM implementing the desired algorithm, and in the second, we execute the generated code. The key observation is that, as the first pass executes at runtime, it has access to secret indices used for arrays and can encode them directly into the generated instructions.

To ensure that the implementation is not vulnerable to timing attacks, both the code generator and the generated code must avoid any timing variations correlated with secret data; to this end, many standard techniques to write constant-time code can be employed. Indeed, we show various algorithms, in the remainder of this section and in Section 4, that perform meaningful operations while eschewing any secret-dependent timing variations.

**Limitations.** Our technique comes with certain limitations and potential issues; we encourage further research to assess their severity and develop possible workarounds.

The available storage space in the CPU register file(s) is fairly limited, on the order of hundreds to few thousands of bits, which restricts the applications of this technique; nevertheless, we demonstrate its applicability to relevant post-quantum cryptosystems in the resource-constrained ARMv7-M architecture, such as all variants of PERK with NIST security levels I and III, as well as NTRU HPS2048509 (NIST security level I).

Larger out-of-order CPUs may struggle with the JIT paradigm, mainly due to the need to shuffle data between level 1 data and instruction caches for code generation and execution, respectively, since these caches are rarely coherent; executing generated code requires a cache invalidation procedure, which may be costly. For other uses of JIT such as in Java, Javascript and .NET runtimes, this is usually not an issue, as code is often generated once and run multiple times, amortizing the cost of cache invalidation. However, our technique is very specific to the array values and indices used, and as such the generated code is run only once. Although we have not investigated this issue in depth for larger CPUs, we theorize that such an issue may be overcome when batch code generation is applicable: first generate code for multiple applications of an algorithm, and later run them. This is the case for the PERK digital signature scheme, which applies many permutation operations in parallel.

Other issues arise from the software side, as out-of-order CPUs usually run full-fledged OSes such as Windows, Linux, macOS, Android, iOS, etc. Due to evolving security threats, recent OS versions restrict code execution outside of “sanctioned” use cases; however, the popularity of JIT runtimes dictates that this should still be possible, and without incurring excessive overhead<sup>14</sup>.

<sup>14</sup> For instance, Apple discusses a workflow to enable JIT compilation in macOS at <https://developer.apple.com/documentation/apple-silicon/porting-just-in-time-compilers-to-apple-silicon>.

A potential issue with out-of-order CPUs, and possibly in-order but superscalar ones, is that of microarchitectural side-channels that are specific to the code generation paradigm. In such CPUs, there is control logic to recognize dependencies between instructions, in order to determine whether a pair of instructions can be scheduled to execute in parallel. This control logic looks at the destination and source registers of instructions<sup>15</sup>, to detect so-called *data hazards* that prevent parallel execution: Read-After-Write (RAW) or *true dependencies*; Write-After-Read (WAR) or *anti-dependencies*; and Write-After-Write (WAW) or *output dependencies*. These refer to the temporal ordering of register reads and writes between earlier and later instructions in the instruction stream.

As our proposed approach recasts secret-indexed array accesses as instructions operating on secret-indexed registers, varying dependency graphs (and hence potential data hazards) are induced by secret data, which may in turn induce timing variations. WAR and WAW hazards may be resolved by the out-of-order machinery through register renaming, although conceptually timing differences may arise in case the virtual register file is oversubscribed; in our opinion, this is a mostly theoretical concern that should be impossible or nearly impossible to exploit in practice. A more serious concern arises from RAW hazards: two copies of the same instruction sequence, but using different register operands, may execute in a different number of cycles. If there is no dependency between these registers, the instruction may execute at the fastest possible rate, whereas dependencies introduce pipeline stalls. Note that such timing variations are not a concern in conventional code (i.e., not generated at runtime): register operands are fixed at compile- or assembly-time, independently of secret data, which results in an equally fixed pattern of pipeline stalls (or lack of them). We note that we have not implemented our technique in out-of-order CPUs, and thus do not have experimental confirmation of this issue; even if confirmed, it is unclear whether it can be exploited to mount a successful side-channel attack.

We stress that the above discussion only applies to more complex CPUs. We have performed extensive experiments in the Cortex-M4, and have been unable to reproduce any instances of timing variations in generated code based on our approach. We credit this to the in-order, single-issue nature of the Cortex-M4, with all instructions of interest (i.e., those which we use to operate on secret data) executing in a single cycle; as such, hazards between them cannot possibly occur, as we have confirmed in practice.

**Building blocks.** In the remainder of this section, we illustrate how to employ our proposed methodology to perform constant-time array accesses (reads and writes) with secret indices. We denote the array length by  $n$ , and while we display algorithms for fixed  $n$  for simplicity, the algorithms are easily adapted to handle differently-sized arrays; indeed, our implementations of Section 4 take  $n$  as a runtime parameter of the code generation functions.

<sup>15</sup> For instructions that interact with the memory subsystem, addresses may also be inspected, but this case is not relevant for the present discussion.

### 3.1 Accessing 32-bit arrays

The simplest possible case consists of an array of elements matching the word size of the target CPU (in the case of the Cortex-M4, 32 bits), and small enough to fit in the integer register file of the CPU. We consider the case of an array of  $n = 8$  elements for simplicity of the code, although in practice, slightly larger arrays still fit on integer registers.

For this scenario, after loading the array into the register file, the core instruction to be synthesized is a `MOV`, where its source register depends on the desired index. We use the 32-bit T3 encoding of the integer register-to-register `MOV` instruction [12, Section A7.7.77], which is displayed in Figure 1. For this initial example, we display in Listing 1.1 C implementations that generate code for a function that performs the desired operation, and call it afterwards.

|    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |   |   |   |   |   |   |    |   |   |   |
|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|---|---|---|---|---|---|----|---|---|---|
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3  | 2 | 1 | 0 |
| 1  | 1  | 1  | 0  | 1  | 0  | 1 | 0 | 0 | 1 | 0 | S | 1 | 1 | 1 | 1 | 0  | 0  | 0  | 0  | Rd |    |   |   | 0 | 0 | 0 | 0 | Rm |   |   |   |

**Fig. 1.** MOV instruction between integer registers. Adapted from [12, Section A7.7.77, encoding T3].

A first note is that writing code as in Listing 1.1, using “magic constants”, is laborious, error-prone and very hard to read and maintain. As such, we provide in our source code package a set of macros that simplify the task of writing code generators, and make them much more readable. For instance, the core of the `array32x8_read` function would look like the following using our macros:

```
APPEND_INSN(PUSH(PUSH_REGRANGE(R(4), R(8))));
APPEND_INSN(LDMIA(R(0), LDM_REGRANGE(R(1), R(8))));
APPEND_INSN(MOV_REG(R(0), R(i + 1)));
APPEND_INSN(POP(POP_REGRANGE(R(4), R(8))));
APPEND_INSN(BX_NOP(LR));
```

In the remainder of the paper, we adopt a higher-level notation which emphasizes the generated assembly code, eschewing the auxiliary setup code, instruction encoding, and the call to the generated function. We illustrate this notation for the same example of 32-bit array reads and writes as Algorithms 3 and 4, respectively. However, this notation may include auxiliary variables computed in the generator (with variables in mathematical italic font, in contrast to the monospaced font used for assembly instructions). Such variables are usually referenced as fields encoded in the generated instructions, e.g., in immediate values and register indices, in which case the notation  $\mathbf{R}[i]$  is used to indicate the register of index  $i$ ; i.e., if  $i = 10$ , then  $\mathbf{R10}$  is accessed.

The general strategy of the functions in Listing 1.1 is to generate code for a function that loads the array into the register file, performs the desired operation,

**Listing 1.1.** C implementations of constant-time 32-bit array accesses

---

```

1 uint32_t array32x8_read(uint32_t* v, int i) { // Returns v[i]
2 // Generated code array; volatile prevents strict aliasing bugs
3 volatile uint32_t code[5];
4
5 // Cast to function pointer, adding 1 to address since it is a
6 // call to a Thumb-encoded function
7 uint32_t (*f)(uint32_t *) =
8 (uint32_t (*)(uint32_t *))(uint8_t *)code + 1);
9
10 code[0] = 0x01f0e92d; // PUSH {R4-R8}
11 code[1] = 0x01fee890; // LDMIA R0, {R1-R8}
12 code[2] = 0x0000ea4f | ((i + 1) << 16); // MOV R0, R[i + 1]
13 code[3] = 0x01f0e8bd; // POP {R4-R8}
14 code[4] = 0xbf004770; // BX LR + NOP
15
16 return f(v);
17 }
18
19 void array32x8_write(uint32_t* v, int i, uint32_t x) { // v[i] = x;
20 volatile uint32_t code[6];
21 void (*f)(uint32_t *, int) =
22 (void (*)(uint32_t *, int))(uint8_t *)code + 1);
23
24 code[0] = 0x03f0e92d; // PUSH {R4-R9}
25 code[1] = 0x03fce890; // LDMIA R0, {R2-R9}
26 code[2] = 0x0001ea4f | ((i + 2) << 24); // MOV R[i + 2], R1
27 code[3] = 0x03fce880; // STMIA R0, {R2-R9}
28 code[4] = 0x03f0e8bd; // POP {R4-R9}
29 code[5] = 0xbf004770; // BX LR + NOP
30
31 f(v, x);
32 }

```

---

and then if necessary writes the register file contents back to the array; the function is then immediately called to perform the desired operation.

Of course, as written, there is no advantage to this approach, due to the cost of loading (and, in the case of writes, storing) the full array from/to the register file; it could instead be implemented by a full scan of the array along with constant-time conditional operations, similar to e.g., [49, Algorithm 4, lines 3–4]. However, in the context of an algorithm that performs such accesses in a loop (assuming  $O(n)$  iterations), the cost of loading and storing the array is amortized across all loop iterations. Therefore, the amortized cost of each array read or write is also  $O(1)$ , just as with a regular timing-unprotected access, although the involved constants are larger; this is the case for Algorithms 7 to 10, described in Section 4.

**Algorithm 3** Constant-time read from 32-bit array.**Input:** Generator: 32-bit array  $v[0, \dots, 7]$ , index  $i$ . Generated code: R0: address of  $v$ .**Output:**  $v[i]$ , returned via R0.

|          |              |                                                   |
|----------|--------------|---------------------------------------------------|
| 1: PUSH  | {R4-R8}      | ▷ Save R4-R8 per AAPCS32                          |
| 2: LDMIA | R0, {R1-R8}  | ▷ Load $v$ to registers R1-R8                     |
| 3: MOV   | R0, R[i + 1] | ▷ Copy $v[i]$ to R0 (return register per AAPCS32) |
| 4: POP   | {R4-R8}      | ▷ Restore R4-R8 per AAPCS32                       |
| 5: BX    | LR           | ▷ Return from function                            |

**Algorithm 4** Constant-time write to 32-bit array: set  $v[i]$  to  $x$ .**Input:** Generator: 32-bit array  $v[0, \dots, 7]$ , index  $i$ , value to be written  $x$ . Generated code: R0: address of  $v$ , R1:  $x$ .

|          |              |                                                                        |
|----------|--------------|------------------------------------------------------------------------|
| 1: PUSH  | {R4-R9}      | ▷ Save R4-R9 per AAPCS32                                               |
| 2: LDMIA | R0, {R2-R9}  | ▷ Load $v$ to registers R2-R9                                          |
| 3: MOV   | R[i + 2], R1 | ▷ Write $x$ to R[i + 2], i.e., the copy of $v[i]$ in the register file |
| 4: STMIA | R0, {R2-R9}  | ▷ Write registers R2-R9 back to $v$                                    |
| 5: POP   | {R4-R9}      | ▷ Restore R4-R9 per AAPCS32                                            |
| 6: BX    | LR           | ▷ Return from function                                                 |

**Constant time considerations.** Recall that the goal of our technique is to avoid array accesses indexed by a secret value, denoted  $i$  in both functions above. More generally, any operation whose execution time varies with this secret index must be avoided to ensure constant-time behavior. We now analyze how the code above depends on  $i$ :

- In the generator, full sequential loads (and stores, in `array32x8_write`) of the array are performed in lines 11, 25 and 27 of Listing 1.1. These are independent of the secret index  $i$ , thus nothing can possibly be leaked. Indeed, the only use of  $i$  is in the generation of `MOV` instructions (lines 12 and 26), where its encoding is obtained by combining with a constant (fixed fields in the instruction encoding) using constant-time operations: additions, shifts by a fixed amount and bitwise OR.
- In the generated code, only the `MOV` instruction depends on  $i$ , for which we assume (per the discussion in Section 2.1) that execution time is independent of the registers being accessed. Thus, although the choice of one of the registers is correlated with  $i$ , no information can be leaked due to the constant-time nature of `MOV`.

Also, note that the generator outputs executable code sequentially to memory, also avoiding any leakage by monitoring the order in which instructions are written to the generated code array. A similar reasoning can be made for the remaining algorithms of this section, and those of Section 4.

### 3.2 Accessing arrays with smaller elements

The arrays to be accessed may have elements smaller than the CPU word size, which is 32 bits for the Cortex-M4. Given the fairly small amount of storage space



in the register file, it is crucial to pack arrays as compactly as possible, using  $k < 32$  bits. For the applications considered in this paper, we mostly use  $k = 8$  bits/element (PERK), except for the permutation application algorithm, or  $k = 2$  bits/element (NTRU). Thus, each register contains multiple array elements. In the following, we refer to each element as a  $k$ -bit *slice* of the corresponding register, and assume that  $k$  is a power of two.

Generating a single MOV instruction is no longer suitable, as it operates over the full 32-bit contents of the target register. For array reads, the full register could be returned from the generated function, and the slice of interest later isolated using shifts and bitwise masks; however, for writes, it is imperative to ensure that the generated code only modifies the target slice and leaves the remaining bits of the register unchanged. The extraction and placement of slices in a register can be performed using only shifts and bitwise masks, which are available in virtually all architectures, but in the particular case of ARMv7-M, a more efficient solution uses the UBFX instruction to extract the desired slice (for reads), and BFI to place it in the desired slice (for writes). Algorithm 5 illustrates how one might implement a constant-time copy of  $v[j]$  to  $v[i]$ , assuming that  $i$  and  $j$  are secret, for an array of  $k$ -bit elements of length  $8 \times 32/k$ .

---

**Algorithm 5** Constant-time copy of one element to another of a  $k$ -bit array: set  $v[i]$  to  $v[j]$ .

---

**Input:** Generator:  $k$ -bit array  $v[0, \dots, 256/k]$ , indices  $i$  and  $j$ , slice width  $k$  in bits.  
 Generated code: R0: address of  $v$ .

|                                                                                |                                                  |
|--------------------------------------------------------------------------------|--------------------------------------------------|
| 1: PUSH {R4-R9}                                                                | ▷ Save R4-R9 per AAPCS32                         |
| 2: LDMIA R0, {R2-R9}                                                           | ▷ Load $v$ to registers R2-R9                    |
| ▷ Aux. variables below computed in generator in CT, using right shifts or ANDs |                                                  |
| 3: $r_i, r_j \leftarrow \lfloor i/(32/k) \rfloor, \lfloor j/(32/k) \rfloor$    |                                                  |
| 4: $p_i, p_j \leftarrow i \bmod (32/k), j \bmod (32/k)$                        |                                                  |
| 5: UBFX R1, R[2 + $r_j$ ], # $p_j$ , # $k$                                     | ▷ Extract $p_j$ -th slice of R[2 + $r_j$ ] to R1 |
| 6: BFI R[2 + $r_i$ ], R1, # $p_i$ , # $k$                                      | ▷ Place R1 in $p_i$ -th slice of R[2 + $r_i$ ]   |
| 7: STMIA R0, {R2-R9}                                                           | ▷ Write registers R2-R9 back to $v$              |
| 8: POP {R4-R9}                                                                 | ▷ Restore R4-R9 per AAPCS32                      |
| 9: BX LR                                                                       | ▷ Return from function                           |

---

### 3.3 Use of floating-point registers for storage

Unfortunately, the integer register file in ARMv7-M is fairly small, with only 14 registers available for use by the generated code, some of which must be used as temporaries for execution of the actual algorithm of interest (e.g., the Fisher-Yates shuffle). Although it is possible to perform such algorithms in a sliced fashion to different bit groups of the array, a fact which we exploit for one of the algorithms needed for PERK, the cost quickly grows as the slice size decreases.

However, as stated in Section 2.4, most commercial parts based on the Cortex-M4 core include hardware support for single-precision floating-point arith-

metic, and in particular, an additional 32-entry register file for a total of 1024 bits of storage. As most cryptographic code eschews the use of floating-point arithmetic, particularly if only single-precision support is available, these registers are available for storage in our approach. Thus, the algorithm can be applied to arrays that are over twice as large, and the integer register file can be freed up for exclusive use of the algorithm of interest.

Algorithms 3 and 4 can be straightforwardly modified to use floating-point instructions such as `VLDMIA`, `VSTMIA` and `VMOV` in place of `LDMIA`, `STMIA` and `MOV`. However, when packing is used as in the case of Algorithm 5, a complication arises, in that floating-point instructions equivalent to `UBFX` and `BFI` (or, in general, bitwise operations which could replace them) are not available. Our strategy is to use the `VMOV` instruction to move data between floating-point and integer register files, so that integer-only operations can be performed on them. The main downside of this strategy is the additional cost to generate and execute such `VMOV` instructions. This approach is illustrated in Algorithm 6, which is a variant of Algorithm 5 using the floating-point register file, and thus capable of handling arrays with up to  $1024/k$  elements. For example, this is sufficient for storing the 508-element ternary array used in fixed-weight sampling for NTRU HPS2048509, or PERK permutations for NIST security levels I and III.

---

**Algorithm 6** Constant-time copy of one element to another of a  $k$ -bit array using floating-point registers: set  $v[i]$  to  $v[j]$ .

---

**Input:** Generator:  $k$ -bit array  $v[0, \dots, 1024/k]$ , indices  $i$  and  $j$ , slice width  $k$  in bits.

Generated code: R0: address of  $v$ .

|                                                                                |                                                                 |
|--------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 1: <code>VPUSH {S16-S31}</code>                                                | ▷ Save S16-S31 per AAPCS32                                      |
| 2: <code>VLDMIA R0, {S0-S31}</code>                                            | ▷ Load $v$ to registers S0-S31                                  |
| ▷ Aux. variables below computed in generator in CT, using right shifts or ANDs |                                                                 |
| 3: $r_i, r_j \leftarrow \lfloor i/(32/k) \rfloor, \lfloor j/(32/k) \rfloor$    |                                                                 |
| 4: $p_i, p_j \leftarrow i \bmod (32/k), j \bmod (32/k)$                        |                                                                 |
| 5: <code>VMOV R1, S[r<sub>j</sub>]</code>                                      | ▷ Copy FP register containing $v[j]$ to R1                      |
| 6: <code>VMOV R2, S[r<sub>i</sub>]</code>                                      | ▷ Copy FP register containing $v[i]$ to R2                      |
| 7: <code>UBFX R1, R1, #p<sub>j</sub>, #k</code>                                | ▷ Extract $p_j$ -th slice of R1 to least signif. bits of itself |
| 8: <code>BFI R2, R1, #p<sub>i</sub>, #k</code>                                 | ▷ Place least significant bits of R1 in $p_i$ -th slice of R2   |
| 9: <code>VMOV S[r<sub>i</sub>], R2</code>                                      | ▷ Copy R2 to FP register containing $v[i]$                      |
| 10: <code>VSTMIA R0, {S0-S31}</code>                                           | ▷ Write registers S0-S31 back to $v$                            |
| 11: <code>VPOP {S16-S31}</code>                                                | ▷ Restore S16-S31 per AAPCS32                                   |
| 12: <code>BX LR</code>                                                         | ▷ Return from function                                          |

---

One might consider a variant that stores part of the array in floating-point registers (say, the first 1024 bits) and the rest in integer registers to maximize the available storage space. Under a very reasonable assumption of having six integer registers available for array storage, it would be possible to implement PERK security level V. Under a more generous assumption of 11 available integer registers, NTRU HPS2048677 fixed-weight sampling could also be implemented.

The main tradeoff with such a variant is additional complexity. The generator code must decide, in constant time, whether the data should be sourced from (and possibly later written to) a floating-point or integer register, using either a `VMOV` or `MOV` instruction, respectively. Both are single-cycle instructions in the Cortex-M4, but their performance may vary in future cores; if one of these instructions performed differently from the other, this would give rise to a timing distinguisher<sup>16</sup>. We have envisioned a workaround, by using a “decoy register” so that, for every iteration, exactly one floating-point and one integer register are read and/or written; the correct one is selected in constant time. As this increases implementation complexity, we leave investigations into the simultaneous use of floating-point and integer registers as future work.

## 4 Secure Permutations Applied to PERK and NTRU

In this section, we present algorithms that perform operations on permutations and fixed-weight sampling, required by the PERK digital signature scheme and the NTRU key encapsulation mechanism. We implement these algorithms using the code generation approach described in Section 3, allowing them to be efficiently carried out in constant time. As we change one of the underlying sampling algorithms in these schemes, we remark that our changes are not compatible with the official KATs. We first recall the relevant background on PERK and NTRU, and then describe the algorithms for the required operations.

### 4.1 Brief overview of PERK and NTRU

**PERK.** The MPCitH signature scheme PERK [1] is built assuming the hardness of the relaxed inhomogeneous Permuted Kernel Problem (r-IPKP). Informally, the r-IPKP is: given a matrix  $\mathbf{H} \in \mathbb{F}_q^{m \times n}$  and pairs of vectors  $(\mathbf{x}_i, \mathbf{y}_i)_{i \in \{t\}} \in \mathbb{F}_q^n \times \mathbb{F}_q^m$ , find a permutation  $\pi \in \mathcal{S}_n$  such that  $\mathbf{H}\pi(\mathbf{x}) = \mathbf{y}$ , where  $\mathbf{x} = \sum_{i \in [t]} \kappa_i \mathbf{x}_i$ ,  $\mathbf{y} = \sum_{i \in [t]} \kappa_i \mathbf{y}_i$ , for some vector  $(\kappa_1, \dots, \kappa_t) \in \mathbb{F}_q^t \setminus \{0, \dots, 0\}$ . For the different NIST security levels PERK-I, PERK-II and PERK-III, we have respectively  $n \in \{79, 83\}$ ,  $n \in \{112, 116\}$ ,  $n \in \{146, 150\}$ , and  $q$  is fixed as the prime 1021. In the commitment phase of the signing algorithm, one has to sample many random permutations (e.g., up to 5120 for PERK-I) and perform several types of operations on them, consisting essentially of inversions, compositions and application to vectors. Sampling permutations and applying them to vectors are performed similarly in the verification algorithm. The submitted reference and optimized implementations of PERK to the NIST standardization process use `djbsort` sorting software [17] to sample random permutations in constant time, as described in Algorithm 2. Sorting is also used in these implementations to compose and invert permutations.

<sup>16</sup> As more general advice, synthesizing different instructions conditional on secret data should be performed with care when using our proposed approach.

**NTRU.** The NTRU key encapsulation mechanism (KEM) [22] is a post-quantum cryptosystem build on top of the NTRU deterministic public key encryption scheme. Its security relies on the difficulty of finding short vectors in lattices of high dimension. Most of the computations take place in polynomial rings  $\mathbb{Z}[x]/(p, \Phi)$ , where  $p \in \{2, 3, q\}$ , with  $q$  a positive power of two, and  $\Phi$  is one of the polynomials  $(x^n - 1)/(x - 1)$  or  $x^n - 1$  for some prime number  $n$ .

In its NTRUHPS variant, the NTRU KEM specification proposes three sets of parameters:  $(n = 509, q = 2048)$ ,  $(n = 677, q = 2048)$  and  $(n = 821, q = 4096)$ , whose corresponding instances are respectively denoted NTRU HPS2048509, NTRU HPS2048677 and NTRU HPS4096821. During the key generation and encapsulation, the scheme requires sampling from the set  $\mathcal{T}(d) \subset \mathbb{Z}[x]/(3, (x^n - 1)/(x - 1))$  of polynomials having  $d/2$  coefficients equal to 1,  $d/2$  coefficients equal to  $-1$  and the rest equal to 0, with  $d = q/4 - 2$ . The natural way to sample from  $\mathcal{T}(d)$  is to start from a fixed polynomial in  $\mathcal{T}(d)$  and then apply a random permutation to its coefficients. Hence, any significant speedup in secure shuffling routines will directly impact the performance of the NTRU key generation and encapsulation processes.

#### 4.2 Secure permutation operations via runtime code generation

All of PERK's permutation operations, i.e., sampling, application, inversion, and composition, are built on the fundamental operations of reading and writing from arrays in constant time described in Section 3. As PERK permutation sizes exceed the available storage space in integer registers, all algorithms are written to use floating-point registers as array storage.

Since the permutation application in PERK and fixed-weight sampling in NTRU constitute simple variants of the fully-described inversion and sampling algorithms, respectively, we just indicate required modifications to implement them. Nonetheless, implementations of all the algorithms are included in our source code package and thus available for inspection by the interested reader.

As every operation described in this section is performed inside loops, the general strategy for all of them is to load a permutation into the register file, loop over it in the generator (while generating unrolled code to be later run), and if necessary, write it back to memory only at the end. Thus, as discussed in Section 3, the cost of loading (and possibly storing) the array is amortized over all loop iterations to a cost of  $O(1)$ . As such, the following algorithms retain their original  $O(n)$  asymptotic performance.

**Permutation inversion.** One of the simplest operations is permutation inversion: given  $\pi \in \mathcal{S}_n$ , represented as a byte array, it computes  $o = \pi^{-1}$  as  $o[\pi[i]] = i$  for  $i = 0, 1, \dots, n-1$ . Our implementation of this operation is displayed in Algorithm 7. As written, it may seem that both  $\pi$  and  $o$  must be held in registers, in which case the available storage space would be insufficient. However, note that only  $o$  is accessed with secret indices;  $\pi$  is accessed sequentially, and thus it would be possible to generate load instructions to  $\pi$  while keeping the code constant

time. We opt to apply a different strategy: it can be seen that the generated code only receives the output permutation  $o$  as a parameter –  $\pi$  is not passed at all. This may appear counterintuitive initially, but note that each element of  $\pi$  is read by the generator and encoded as part of the instructions that populate  $o$  when the generated code is run. This explains why  $\pi$  is not directly used by the generated code, and thus only the storage space for a single array  $o$  is required. Moreover, as  $o$  is uninitialized when passed to the algorithm, and is completely populated during its execution, it is not necessary to load  $o$  into the register file at the beginning of the algorithm.

---

**Algorithm 7** Permutation inversion.

---

**Input:** Generator: a permutations  $\pi \in \mathcal{S}_n$ . Generated code: R0: address of a byte array  $o$  of length  $n$ .

**Output:** The inverse permutation  $o = \pi^{-1}$ .

```

1: VPUSH {S16-S[[(n-1)/4]1]} ▷ Save FP registers S16 onwards per AAPCS32
2: for $i = 0$ to $n - 1$ do ▷ Loop in generator; generated code must be unrolled
3: $i_\pi, b_\pi \leftarrow \lfloor \pi[i]/4 \rfloor, \pi[i] \bmod 4$ ▷ Auxiliary variables computed in generator
4: VMOV R1, S[i_π] ▷ Copy FP register containing $o[\pi[i]]$
5: MOVW R2, # i ▷ Load constant i
6: BFI R1, R2, # b_π , #8 ▷ Set b_π -th byte of R1 to i
7: VMOV S[i_π], R1 ▷ Save to FP register containing $o[\pi[i]]$
8: VSTMIA R0, {S0-S[[(n-1)/4]1]} ▷ Store FP registers back to o
9: VPOP {S16-S[[(n-1)/4]1]} ▷ Restore FP registers S16 onwards per AAPCS32
10: BX LR ▷ Return from function

```

---

Otherwise, the main loop of the implementation is simple: the floating-point register containing  $o[\pi[i]]$  and neighboring elements is copied to a temporary integer register (R1) in line 4;  $i$  is loaded, as an immediate, into another temporary (R2) in line 5 using the MOVW instruction; BFI places  $i$  into the appropriate byte of R1 in line 6; and finally, R1 is stored back to the floating-point register containing  $o[\pi[i]]$ . Note that all accesses to  $p$  in the generator are performed sequentially, and thus do not leak any information about the secret indices. In the generated code, the full array is loaded sequentially at the beginning and written back also sequentially at the end, again not leaking any information about the secret indices; out-of-order accesses to  $o[\pi[i]]$  are made using VMOV instructions, and thus are constant-time.

**Permutation composition with inverse.** The next operation we consider is permutation composition with inverse: given  $\pi_1, \pi_2 \in \mathcal{S}_n$ , we compute  $\pi = \pi_1 \circ \pi_2^{-1}$ . It is implemented as  $\pi[\pi_2[i]] = \pi_1[i]$ , which implementation-wise is almost identical to the permutation inversion; the only difference is that the value stored is  $\pi_1[i]$  rather than  $i$ . Other than renaming variables, we make a single change in line 5 of Algorithm 7 to obtain Algorithm 8: we load  $\pi_1[i]$  rather than  $i$ . Secret data leakage is not a concern as  $\pi_1$  is accessed sequentially.

**Algorithm 8** Permutation composition with inverse.

**Input:** Generator: permutations  $\pi_1, \pi_2 \in \mathcal{S}_n$ . Generated code: R0: address of a byte array  $\pi$  of length  $n$ .

**Output:**  $\pi = \pi_1 \circ \pi_2^{-1}$ .

```

1: VPUSH {S16-S[(n-1)/4]} ▷ Save FP registers S16 onwards per AAPCS32
2: for $i = 0$ to $n - 1$ do ▷ Loop in generator; generated code must be unrolled
3: $i_{\pi_2}, b_{\pi_2} \leftarrow \lfloor \pi_2[i]/4 \rfloor, \pi_2[i] \bmod 4$ ▷ Auxiliary variables computed in generator
4: VMOV R1, S[i_{π_2}] ▷ Copy FP register containing $\pi_2[i]$
5: MOVW R2, # $\pi_1[i]$ ▷ Load constant $\pi_1[i]$
6: BFI R1, R2, # b_{π_2} , #8 ▷ Set b_{π_2} -th byte of R1 to $\pi_1[i]$
7: VMOV S[i_{π_2}], R1 ▷ Save to FP register containing $\pi_2[i]$
8: VSTMIA R0, {S0-S[(n-1)/4]} ▷ Store FP registers back to π
9: VPOP {S16-S[(n-1)/4]} ▷ Restore FP registers S16 onwards per AAPCS32
10: BX LR ▷ Return from function

```

**Permutation application.** Another operation required by PERK is permutation application: an input 16-bit array  $v$  of length  $n$  must be permuted according to  $\pi \in \mathcal{S}_n$ , writing the output to a second 16-bit array  $o$ . The operation can be described as  $o[\pi[i]] = v[i]$  for  $i = 0, 1, \dots, n - 1$ . It is essentially identical to permutation composition with inverse, and so if  $o$  and  $v$  were 8-bit arrays, Algorithm 8 could be used directly.

However, since the arrays have 16-bit elements, a problem arises: there is insufficient storage space in the floating-point register file. Indeed, even including all 14 integer registers would prove insufficient for NIST level III. We resolve this conundrum by noting that permutations can be independently applied, in two passes, to the least significant and most significant bytes of the 16-bit arrays. Also, we depart from the strategy of Algorithm 8 of loading  $\pi_1[i]$  (denoted by  $v[i]$  in this case) as an immediate in line 5; instead, we also pass  $v$  as an argument to the generated function, and load  $v[i]$  directly from it<sup>17</sup>. Thus, we can generate the function only once, but call it twice (for each array containing the lower and upper half of the 16-bit array elements), halving the cost of code generation.

For this to work, however, only the least or most significant byte of each element must be stored in each pass, precluding the generation of a single instruction (VSTMIA) for storing the array in line 8. A naive solution is to generate instructions to copy each floating-point register to a temporary integer register using VMOV, extract each byte (either the least or most significant one of each element) using UBFX, and store that byte using STRB. The overhead of such byte-by-byte processing would be very significant, and thus we opt to change the representation of the array. Before calling the generated code, we apply a byte-wise perfect unshuffle operation to deinterleave the least significant and most significant bytes of each element into two separate arrays. Next, we run the generated code twice to apply the permutation to each of these arrays, and finally interleave the least significant and most significant bytes using a byte-

<sup>17</sup> Note that such a direct load from memory does not leak any secret information, since  $v$  is loaded sequentially from memory.

wise perfect shuffle operation. Interleaving and deinterleaving are implemented very efficiently using instructions from the Cortex-M4 DSP extensions, although shifts and bitwise instructions could be used instead, with a small performance impact, to support ARM cores without this extension or other architectures.

**Permutation composition.** The next operation required for PERK is permutation composition: Given  $\pi_1, \pi_2 \in \mathcal{S}_n$ , compute  $\pi = \pi_1 \circ \pi_2$ , which can be implemented as  $\pi[i] = \pi_1[\pi_2[i]]$  for  $i = 0, 1, \dots, n-1$ . Note that  $\pi$  and  $\pi_2$  are accessed sequentially, and thus only  $\pi_1$  needs to be stored in registers to avoid leakage. This operation is implemented by Algorithm 9. The main novelty with regard to the previous algorithms is that we synthesize a store instruction (STRB) in line 7 to write to the output array  $\pi$  directly; this eschews the need to write the floating-point registers back to memory.

---

**Algorithm 9** Permutation composition.

---

**Input:** Generator: permutations  $\pi_1, \pi_2 \in \mathcal{S}_n$ . Generated code: R0: address of a byte array  $\pi$  of length  $n$ , R1: address of  $\pi_1$ , represented as a byte array of length  $n$ .

**Output:**  $\pi = \pi_1 \circ \pi_2$ .

```

1: VPUISH {S16-S[(n-1)/4]} ▷ Save FP registers S16 onwards per AAPCS32
2: VLDMIA R0, {S0-S[(n-1)/4]} ▷ Load π_1 to FP registers
3: for $i = 0$ to $n-1$ do ▷ Loop in generator; generated code must be unrolled
4: $i_{\pi_2}, b_{\pi_2} \leftarrow \lfloor \pi_2[i]/4 \rfloor, \pi_2[i] \bmod 4$ ▷ Auxiliary variables computed in generator
5: VMOV R2, S[i_{π_2}] ▷ Copy FP register containing $\pi_2[i]$
6: UBFX R2, R2, # b_{π_2} , #8 ▷ Shift b_{π_2} -th byte of R2 to its least significant byte
7: STRB R2, [R0, # i] ▷ Store R2 to $\pi[i]$
8: VPOP {S16-S[(n-1)/4]} ▷ Restore FP registers S16 onwards per AAPCS32
9: BX LR ▷ Return from function

```

---

**Permutation sampling.** Now we turn to the issue of sampling a permutation, which can be performed using a variant of the Fisher-Yates shuffle. Listing 1.2 displays the C code for the function that performs permutation sampling in a timing-unprotected implementation of PERK, where `PARAM_N1` corresponds to the array length  $n$ . It is seen to be similar to Algorithm 1, but slightly rewritten so as to eschew the need to initialize the permutation array `p` (and therefore the initial load to the register file in our implementation). The `rnd_buff` array is sampled according to line 3 of Algorithm 1, using a fast and constant-time technique due to Lemire [38], which does not require division instructions.

Algorithm 10 displays our implementation of Listing 1.2 using the code generation approach. Although it is, at a high level, a straightforward translation of the unprotected implementation, care must be taken in the case that  $i_i = i_r$ , i.e., when the same floating-point register contains  $\pi[i]$  and  $i$  (which is true in particular if  $\pi[i] = i$ ). If the code were to keep copies of both `S[ $i_i$ ]` and `S[ $i_r$ ]` in different integer registers and update them separately, then, when writing back

**Listing 1.2.** Timing-unprotected C implementation of permutation sampling

---

```

1 int sig_perk_perm_gen_given_random_input(
2 perm_t p, const uint16_t rnd_buff[PARAM_N1]) {
3 for (int i = 0; i < PARAM_N1; i++) {
4 p[i] = p[rnd_buff[i]];
5 p[rnd_buff[i]] = i;
6 }
7 return EXIT_SUCCESS;
8 }

```

---

both copies to floating-point registers, one of the updates would be shadowed by the other. This introduces a correctness error analogous to a race condition in a parallel program.

To address this issue, we compute a variable  $t$  in the code generator, which is set to 0 if  $i_i = i_r$  and 1 otherwise; this computation is easily performed in constant time. We copy  $S[i_i]$  to R1 in line 6 and  $S[i_r]$  to R[1+t] in line 7; thus, if  $t = 0$  (i.e.,  $i_i = i_r$ ), they are copied to the same register (R1), but otherwise they are copied to different registers (R1 and R2). The same idea is used when copying the updated version to  $S[i_i]$  in line 12 and  $S[i_r]$  in line 13, thus ensuring that the generated code is both correct and constant time.

---

**Algorithm 10** Permutation sampling using Fisher-Yates, baseline version.

**Input:** Generator: an array of random integers  $r$  of length  $n$ , such that  $r[i] \leftarrow \mathcal{U}([i])$ .

Generated code: R0: address of a byte array  $\pi$  of length  $n$ .

**Output:** A random permutation  $\pi \in \mathcal{S}_n$ .

```

1: VPUSH {S16-S[(n-1)/4]} ▷ Save FP registers S16 onwards per AAPCS32
2: for i = 0 to n - 1 do ▷ Loop in generator; generated code must be unrolled
3: $i_i, b_i \leftarrow \lfloor i/4 \rfloor, i \bmod 4$ ▷ Auxiliary variables computed in generator
4: $i_r, b_r \leftarrow \lfloor r[i]/4 \rfloor, r[i] \bmod 4$ ▷ Auxiliary variables computed in generator
5: $t \leftarrow 0$ if $i_i = i_r$, else 1 ▷ Auxiliary variable computed in CT in generator
6: VMOV R1, S[i_i] ▷ Copy FP register containing $\pi[i]$ to R1
7: VMOV R[1+t], S[i_r] ▷ Copy FP register containing $r[i]$ to R1 or R2
8: UBFX R3, R[1+t], #b_r, #8 ▷ Copy b_r -th byte of R1 or R2 to R3
9: BFI R1, R3, #b_i, #8 ▷ Set b_i -th byte of R1 to the value of R3
10: MOVW R3, #i ▷ Load constant i
11: BFI R[1+t], R3, #b_r, #8 ▷ Set b_r -th byte of R[1+t] to i
12: VMOV S[i_i], R1 ▷ Save to FP register containing $\pi[i]$
13: VMOV S[i_r], R[1+t] ▷ Save to FP register containing $r[i]$
14: VSTMIA R0, {S0-S[(n-1)/4]} ▷ Store FP registers back to π
15: VPOP {S16-S[(n-1)/4]} ▷ Restore FP registers S16 onwards per AAPCS32
16: BX LR ▷ Return from function

```

---



Algorithm 10 is labeled as a “baseline version”, since our actual implementation includes some performance improvements which we omit from this description, although they are available for perusal in our source code package. The two main improvements are:

1. The initial 48 iterations of the loop use only integer, rather than floating-point, registers. Note that 12 integer registers can be dedicated to array storage, and that the  $i$ -th iteration is limited to accessing the first  $i$  entries of the permutation array. Fewer instructions are required as no `VMOV` instructions must be synthesized. At the end of the 48th iteration, these 12 integer registers are saved to memory and then reloaded to floating-point registers.
2. The loop body handles four iterations at a time, rather than one. Note that, in a sequence of four iterations starting with  $i \equiv 0 \pmod{4}$ , the value of  $i_i$  in line 3 of Algorithm 10 is identical for all four iterations. Therefore, it is only necessary to generate the `VMOV` instructions of lines 6 and 12 once for each block of four iterations. Therefore, only  $2 + 6 \times 4 = 26$  instead of  $8 \times 4 = 32$  instructions need to be generated for each block of four iterations.

**Fixed-weight sampling for NTRU.** The final operation we consider is fixed-weight sampling for NTRU. It can be implemented by initializing a vector with the prescribed number of 0, 1, and  $-1$  elements and randomly permuting it, using the Fisher-Yates shuffle. Our implementation closely follows Algorithm 1, while implementing the improvements just discussed for permutation sampling: avoiding a correctness issue when  $i_i = i_r$ ; use of integer registers when possible; handling four iterations of the loop body at a time to reduce the number of generated instructions.

## 5 Performance Results

**Benchmarking methodology.** To study the impact of Algorithms 7 to 10, we integrate them into the implementations of PERK and NTRU from the `pqm4` framework [31, 32], and conduct experiments on an STMicroelectronics NUCLEO-L4R5ZI development board. The compiler is `gcc 13.2`. All schemes’ variants are tested using their `test.py` and `testvectors.py` scripts to verify correctness. In particular, the reference implementation used to generate test vectors using the `testvectors.py` script is based on unprotected Fisher-Yates and permutation operations (using array accesses indexed by secret data), therefore ensuring that our code generation implementation produces identical results to these unprotected implementations. Benchmarks are compiled with speed optimizations, and we report average cycle counts for 10 runs of each combination of scheme, variant, and security level.

We perform some configurations of the target system to eliminate slowdowns during the execution of generated code from SRAM, which we detail in Appendix A, where we also quantify the effect of foregoing such configurations.

All performance figures listed next, for implementations based on our approach, employ such configurations to attain maximum generated code performance.

In all tables in this section, the implementations employing the techniques introduced in Sections 3 and 4 are referred to as **codegen**, and their benchmarks include the cost of both code generation and execution of the generated code. Additionally, to compare them with the presumably best achievable performance, we report in *gray* the benchmarks corresponding to non-constant-time instantiations of the same algorithms. The speed-up and percentages improvement are given with respect to the reference implementation.

### 5.1 Microbenchmarks

We display microbenchmarks for the permutation operations discussed in Section 4, using the same parameters (array lengths) as the PERK submission for NIST security levels I and III, in Table 2. We also report fixed-weight sampling performance for  $n = 508$ , as used in NTRU HPS2048509, in Table 3. Each benchmark is run only once with fixed inputs; due to the simple, deterministic nature of the Cortex-M4 core, cycle counts are identical across multiple runs.

We consider the following implementation strategies: constant-time sorting, secret-indexed array accesses, and our code generation approach, and additionally the algorithm of [27] for NTRU. Figures include the cost of setting up and extracting data from arrays for the sorting approach, and Lemire sampling for permutation and fixed-weight sampling. However, we exclude the cost of sampling pseudorandom bits, which varies widely depending on the symmetric primitive chosen for such sampling, which is additionally scheme- and/or platform-dependent. We estimate that, for PERK, the amount of randomness required by the different approaches for permutation sampling is similar: both consume a minimum of 16 bits per element, and rejection probabilities for Lemire sampling (per the analysis of [27, p. 10]) or the sorting approach (in case of collisions in the sorting keys, which can be analyzed using a birthday paradox argument) are in the range of 5–10%. An additional issue is that collisions in the sorting approach require rerunning the algorithm, but we have disabled this behavior in our microbenchmarks; if this were taken into account, a birthday paradox argument suggests that the cycle counts of the sorting approach in Table 2 would increase by approximately 4.8–10.7%, depending on security level, on average.

As for NTRU, the sorting approach consumes approximately 78% more random bits for the considered parameter set, but the sorting approach does not check for collisions. Thus, the figures in Table 3 also underestimate the performance advantage of non-sorting approaches, but only due to the cost of pseudo-randomness sampling.

### 5.2 Performance for PERK

To evaluate the impact of our contribution, we integrate our work into the dedicated implementation of PERK for resource-constrained devices [19]. We replace all permutation-related routines with our code generation variants, Algorithms 7

**Table 2.** Microbenchmarks for Algorithms 7 to 9, the optimized version of Algorithm 10, and vector permutation against their variants based on either non-constant-time secret-indexed array accesses (which we denote as “Table lookup”) or constant-time sorting. The parameter  $n$  is the length of the permutations/vectors. For **sampling**, we exclude from these benchmarks the cost of the pseudorandomness sampling. Performance figures are in clock cycles, and speedup is calculated as the ratio of reference to codegen cycle counts.

| $n$ | Method         | compose     | comp_inv    | sampling    | inverse     | vec_permute |
|-----|----------------|-------------|-------------|-------------|-------------|-------------|
| 79  | Table lookup   | 890         | 1648        | 2345        | 1569        | 1456        |
|     | <b>Codegen</b> | <b>1702</b> | <b>2280</b> | <b>4911</b> | <b>2186</b> | <b>3409</b> |
|     | Reference [19] | 32456       | 16068       | 17686       | 16270       | 16158       |
|     | Speedup        | 19×         | 7.0×        | 3.6×        | 7.4×        | 4.7×        |
| 83  | Table lookup   | 940         | 1698        | 2468        | 1632        | 1512        |
|     | <b>Codegen</b> | <b>1812</b> | <b>2393</b> | <b>5225</b> | <b>2237</b> | <b>3586</b> |
|     | Reference [19] | 34488       | 17215       | 18825       | 17286       | 17237       |
|     | Speedup        | 19×         | 7.2×        | 3.6×        | 7.7×        | 4.8×        |
| 112 | Table lookup   | 2071        | 2235        | 3375        | 2164        | 2038        |
|     | <b>Codegen</b> | <b>2367</b> | <b>3319</b> | <b>7564</b> | <b>2817</b> | <b>4941</b> |
|     | Reference [19] | 49456       | 24790       | 26760       | 24783       | 24734       |
|     | Speedup        | 21×         | 7.5×        | 3.5×        | 8.8×        | 5.0×        |
| 116 | Table lookup   | 2139        | 2315        | 3440        | 2296        | 2057        |
|     | <b>Codegen</b> | <b>2450</b> | <b>3434</b> | <b>7883</b> | <b>2916</b> | <b>5107</b> |
|     | Reference [19] | 51628       | 25783       | 27930       | 25759       | 25700       |
|     | Speedup        | 21×         | 7.5×        | 3.5×        | 8.8×        | 5.0×        |

to 10, and measure the resulting performance improvements. Comparative results are presented in Tables 4 and 5, corresponding to the parameter sets for security levels I and III. Additionally, we provide benchmark results using only the naïve Fisher–Yates shuffle and standard permutation algorithms, in order to have a plausible baseline for optimal performance without side-channel protection. It can be observed that our code generation approach is significantly faster than the sorting-based method used in [19], and achieves performance close to that of the naïve Fisher–Yates implementation. This improvement is particularly notable in the signing operation, where it yields speedups in the range of 33–35%. The trade-off, as expected, is an increase in RAM usage and code size.

### 5.3 Performance for NTRU

Table 6 showcases the efficiency of our code generation approach in the context of NTRU. Analogously to PERK, we report benchmarks obtained by using naïve Fisher–Yates for comparison. We also include the performance of the constant-

**Table 3.** Microbenchmarks for NTRU’s fixed-weight sampling routine, exclusive of the cost for pseudorandomness sampling. The parameter  $n$  is the length of the vectors. Performance figures are in clock cycles, and speedup is calculated as the ratio of CT sort to codegen cycle counts.

| $n$ | Method          | sample.fixed.weight |
|-----|-----------------|---------------------|
| 508 | CT shuffle [27] | 13354               |
|     | Fisher-Yates    | 16834               |
|     | <b>Codegen</b>  | <b>44003</b>        |
|     | CT sort         | 163383              |
|     | Speedup         | 3.7×                |

time sorting-based method (implemented using the `djb`sort library), which is the default in the NTRU implementation, and the specific-purpose shuffle method from [27]. The results show that our method provides a significant speedup over the similarly non-specific sorting approach, maintaining a reasonable code size. It is worth noting that our method also applies to other variants of NTRU, such as those of NTRU Prime [22], which have similar requirements for sampling fixed-weight polynomials.

## 6 Conclusion

This work introduces a novel approach for implementing, in constant time, algorithms that inherently require array accesses indexed by secret data, without altering their asymptotic time complexity. We apply our method to permutation-related operations commonly used in cryptography, including sampling, composition, and inversion. Our experiments show that this technique outperforms existing constant-time alternatives, significantly narrowing the performance gap with the most efficient, but non-constant-time, implementations. When integrated into the post-quantum schemes PERK and NTRU, our method yields notable performance improvements over the alternative based on constant-time sorting; it is only incapable of overcoming a constant-time algorithm that efficiently exploits the specific structure of fixed-weight sampling for NTRU [27].

### 6.1 Future work

We envision many opportunities to extend the applicability of our technique.

**Application to larger arrays.** At the end of Section 3, we have alluded to the possibility of simultaneously using integer and floating-point registers for array storage, but cautioned about additional complexity. Nevertheless, this extends the applicability of our techniques to higher security levels such as PERK-V and NTRU HPS2048677, and given that alternatives may have higher asymptotic

**Table 4.** PERK-128 (NIST security level I) key generation, signing and verification performance and RAM usage on the NUCLEO-L4R5ZI board, using different methods for implementing permutations. Performance figures are in millions of clock cycles, averaged over 10 runs, and sizes are in KB. Speedup is calculated as the ratio of reference to codegen cycle counts, and size impact as the ratio of codegen to reference.

| Variant | Method                | Keygen       |             | Sign       |             | Verify      |             | Code size   |
|---------|-----------------------|--------------|-------------|------------|-------------|-------------|-------------|-------------|
|         |                       | Perf.        | RAM         | Perf.      | RAM         | Perf.       | RAM         |             |
| fast-3  | Table lookup          | 0.579        | 7.62        | 138        | 24.0        | 66.5        | 20.7        | 12.7        |
|         | <b>Codegen</b>        | <b>0.594</b> | <b>9.25</b> | <b>153</b> | <b>25.7</b> | <b>68.8</b> | <b>22.3</b> | <b>14.6</b> |
|         | Ref. [19]             | 0.644        | 7.71        | 245        | 24.0        | 81.1        | 20.7        | 11.6        |
|         | Speedup / size impact | 1.08×        | 20.0%       | 1.60×      | 7.08%       | 1.18×       | 7.73%       | 26%         |
| fast-5  | Table lookup          | 0.715        | 9.03        | 132        | 25.2        | 63.4        | 21.8        | 12.7        |
|         | <b>Codegen</b>        | <b>0.732</b> | <b>10.8</b> | <b>145</b> | <b>26.9</b> | <b>65.7</b> | <b>23.5</b> | <b>14.6</b> |
|         | Ref. [19]             | 0.823        | 9.03        | 240        | 25.2        | 78.1        | 21.8        | 11.6        |
|         | Speedup / size impact | 1.12×        | 19.6%       | 1.66×      | 6.75%       | 1.19×       | 7.80%       | 25.9%       |
| short-3 | Table lookup          | 0.579        | 7.62        | 747        | 27.8        | 378         | 25.2        | 25.6        |
|         | <b>Codegen</b>        | <b>0.589</b> | <b>9.25</b> | <b>830</b> | <b>29.4</b> | <b>392</b>  | <b>26.9</b> | <b>27.5</b> |
|         | Ref. [19]             | 0.641        | 7.71        | 1321       | 27.8        | 461         | 25.2        | 24.5        |
|         | Speedup / size impact | 1.09×        | 20.0%       | 1.59×      | 5.76%       | 1.18×       | 6.75%       | 12.2%       |
| short-5 | Table lookup          | 0.716        | 9.03        | 694        | 28.6        | 348         | 26.1        | 25.7        |
|         | <b>Codegen</b>        | <b>0.732</b> | <b>10.8</b> | <b>763</b> | <b>30.4</b> | <b>361</b>  | <b>27.8</b> | <b>27.5</b> |
|         | Ref. [19]             | 0.823        | 9.03        | 1245       | 28.6        | 428         | 26.1        | 24.6        |
|         | Speedup / size impact | 1.12×        | 19.6%       | 1.63×      | 6.29%       | 1.19×       | 6.51%       | 11.8%       |

complexity (e.g.,  $O(n \log^2 n)$  for constant-time sorting), our technique may still outperform alternatives despite increased constant factors.

Another strategy to work around storage limits was discussed in Section 4, in the context of permutation application: one may apply permutation operations in multiple passes, to different “slices” of each element of the permutation array (in that particular case, 16-bit elements were split into their least and most significant bytes). For instance, by separately shuffling each of the two bits in packed NTRU fixed-weight sampling arrays, our approach can be extended to NTRU HPS4096821. When both strategies are applicable, it is unclear which of the two would perform better; we encourage investigations into the relative performance of both.

**Application to more schemes.** While we have demonstrated the validity of our technique for PERK and NTRU, other applications could benefit from it. For instance, the LESS post-quantum signature scheme [14] employs an unprotected version of the Fisher-Yates shuffle; our technique would increase its

**Table 5.** PERK-192 (NIST security level III) key generation, signing and verification performance and RAM usage on the NUCLEO-L4R5ZI board, using different methods for implementing permutations. Performance figures are in millions of clock cycles, averaged over 10 runs, and sizes are in KB. Speedup is calculated as the ratio of reference to codegen cycle counts, and size impact as the ratio of codegen to reference.

| Variant | Method                | Keygen      |             | Sign        |             | Verify     |             | Code size   |
|---------|-----------------------|-------------|-------------|-------------|-------------|------------|-------------|-------------|
|         |                       | Perf.       | RAM         | Perf.       | RAM         | Perf.      | RAM         |             |
| fast-3  | Table lookup          | 1.39        | 14.8        | 322         | 47.7        | 157        | 41.4        | 12.3        |
|         | <b>Codegen</b>        | <b>1.41</b> | <b>17.3</b> | <b>357</b>  | <b>50.2</b> | <b>163</b> | <b>43.8</b> | <b>14.6</b> |
|         | Ref. [19]             | 1.49        | 15.0        | 582         | 47.8        | 195        | 41.4        | 12.0        |
|         | Speedup / size impact | 1.06×       | 15.3%       | 1.63×       | 5.02%       | 1.20×      | 5.80%       | 21.7%       |
| fast-5  | Table lookup          | 1.65        | 16.9        | 306         | 48.8        | 150        | 42.4        | 12.3        |
|         | <b>Codegen</b>        | <b>1.68</b> | <b>19.4</b> | <b>337</b>  | <b>51.3</b> | <b>156</b> | <b>45.0</b> | <b>14.6</b> |
|         | Ref. [19]             | 1.80        | 16.9        | 560         | 48.8        | 187        | 42.4        | 11.9        |
|         | Speedup / size impact | 1.07×       | 14.8%       | 1.66×       | 5.12%       | 1.20×      | 6.13%       | 22.7%       |
| short-3 | Table lookup          | 1.39        | 14.8        | 1829        | 51.3        | 964        | 46.7        | 24.3        |
|         | <b>Codegen</b>        | <b>1.41</b> | <b>17.3</b> | <b>2030</b> | <b>53.8</b> | <b>998</b> | <b>49.1</b> | <b>26.6</b> |
|         | Ref. [19]             | 1.51        | 15.0        | 3281        | 51.5        | 1179       | 46.7        | 23.9        |
|         | Speedup / size impact | 1.07×       | 15.3%       | 1.62×       | 4.47%       | 1.18×      | 5.14%       | 11.3%       |
| short-5 | Table lookup          | 1.65        | 16.9        | 1687        | 51.9        | 894        | 47.3        | 24.9        |
|         | <b>Codegen</b>        | <b>1.68</b> | <b>19.4</b> | <b>1868</b> | <b>54.5</b> | <b>926</b> | <b>49.8</b> | <b>27.2</b> |
|         | Ref. [19]             | 1.81        | 16.9        | 3045        | 51.9        | 1101       | 47.3        | 24.6        |
|         | Speedup / size impact | 1.08×       | 14.8%       | 1.63×       | 5.01%       | 1.19×      | 5.29%       | 10.6%       |

side-channel resistance. Going beyond post-quantum cryptosystems, symmetric schemes could benefit from constant-time table lookups, a prime example being AES. Given the limited storage space in registers, the use of full T-tables is impractical, and even the S-boxes are too large to be directly used; however, this can be overcome, using floating-point registers only, by applying the strategy of the previous paragraph, i.e., splitting each element of the cipher state into 4-bit halves and performing lookups in two passes.

**Extending to other architectures.** A natural extension of our work is applying it to other architectures. RISC-V has been gaining traction in the embedded space, and appears well-suited to the use of our technique, due to its RISC philosophy, with simple, regular instruction encodings and a large integer register file. Larger out-of-order CPUs might be investigated as well, although the caveats discussed in Section 3 should be kept in mind; in particular, a robust benchmarking methodology should be employed to account for the costs of

**Table 6.** NTRU HPS2048509 (NIST security level I) performance and RAM usage for key generation and encapsulation on the NUCLEO-L4R5ZI board, for different fixed-weight sampling methods. Decapsulation performance is not reported, as it does not employ fixed-weight sampling. Performance figures are in thousands of clock cycles, averaged over 10 runs, and sizes are in KB. Speedup is calculated as the ratio of CT sort to codegen cycle counts, and size impact as the ratio of codegen to reference.

| Sampling method       | Key generation |             | Encapsulation |             | Code size  |
|-----------------------|----------------|-------------|---------------|-------------|------------|
|                       | Perf.          | RAM         | Perf.         | RAM         |            |
| CT shuffle [27]       | 2665           | 20.5        | 240           | 13.2        | 192        |
| Fisher-Yates          | 2670           | 20.5        | 245           | 13.2        | 192        |
| <b>Codegen</b>        | <b>2701</b>    | <b>31.3</b> | <b>276</b>    | <b>24.0</b> | <b>198</b> |
| CT sort               | 2834           | 21.3        | 409           | 14.0        | 192        |
| Speedup / size impact | 1.05×          | 46.9%       | 1.48×         | 71.4%       | 3.12%      |

cache invalidation, and its constant-time properties should be thoroughly validated prior to recommending its use.

**Optimization of the code.** Lastly, we envision further optimizations to our technique, which may reduce constant factors in its application. While many instructions depend on values that are only known at runtime (and indeed, that is the main point of our technique), others do not; for instance, lines 6, 10, 9 and 12 of algorithm 10 depend only on the loop variable  $i$ . Thus, the generated code array can be initialized with the (fixed) instruction encodings corresponding to these lines for all iterations of the loop, reducing the amount of work that must be performed during code generation. Yet another possibility to investigate is moving away from the two-pass process (code generation and generated code execution) to a self-modifying code paradigm; that is, an instruction may directly modify the following one in the instruction stream, which is then immediately executed. This presents challenges (register pressure; foregoing the possibility of writing the generator code in a high-level language; unfriendliness to caches and prefetch queues) but also potential benefits, mainly in terms of eliminating overhead, such as generating code for function prologue and epilogue, loading and storing arrays to/from the register files and other instructions that are independent of runtime values.

## References

1. Aaraj, N., Bettaieb, S., Bidoux, L., Budroni, A., Dyseryn, V., Esser, A., Feneuil, T., Gaborit, P., Kulkarni, M., Mateu, V., Palumbi, M., Perin, L., Rivain, M., Tillich, J., Xagawa, K.: PERK. Tech. rep., National Institute of Standards and Technology (2024), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>

2. Abdulrahman, A., Chen, J.P., Chen, Y.J., Hwang, V., Kannwischer, M.J., Yang, B.Y.: Multi-moduli NTTs for Saber on Cortex-M3 and Cortex-M4. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2022**(1), 127–151 (2022). <https://doi.org/10.46586/tches.v2022.i1.127-151>
3. Abdulrahman, A., Hwang, V., Kannwischer, M.J., Sprenkels, A.: Faster Kyber and Dilithium on the Cortex-M4. In: Ateniese, G., Venturi, D. (eds.) *ACNS 22: 20th International Conference on Applied Cryptography and Network Security*. Lecture Notes in Computer Science, vol. 13269, pp. 853–871. Springer, Cham (Jun 2022). [https://doi.org/10.1007/978-3-031-09234-3\\_42](https://doi.org/10.1007/978-3-031-09234-3_42)
4. Adomnica, A., Peyrin, T.: Fixslicing AES-like ciphers. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2021**(1), 402–425 (2021). <https://doi.org/10.46586/tches.v2021.i1.402-425>, <https://tches.iacr.org/index.php/TCHES/article/view/8739>
5. Advanced Encryption Standard (AES). National Institute of Standards and Technology, NIST FIPS PUB 197, U.S. Department of Commerce (Nov 2001)
6. Albrecht, M.R., Bernstein, D.J., Chou, T., Cid, C., Gilcher, J., Lange, T., Maram, V., von Maurich, I., Misoczki, R., Niederhagen, R., Paterson, K.G., Persichetti, E., Peters, C., Schwabe, P., Sendrier, N., Szefer, J., Tjhai, C.J., Tomlinson, M., Wang, W.: Classic McEliece. Tech. rep., National Institute of Standards and Technology (2022), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-4-submissions>
7. Alkim, E., Cheng, D.Y.L., Chung, C.M.M., Evkan, H., Huang, L.W.L., Hwang, V., Li, C.L.T., Niederhagen, R., Shih, C.J., Wälde, J., Yang, B.Y.: Polynomial multiplication in NTRU prime. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2021**(1), 217–238 (2021). <https://doi.org/10.46586/tches.v2021.i1.217-238>, <https://tches.iacr.org/index.php/TCHES/article/view/8733>
8. Anastasova, M., Azarderakhsh, R., Kermani, M.M., Beshaj, L.: Time-efficient finite field microarchitecture design for Curve448 and Ed448 on cortex-M4. In: Seo, S.H., Seo, H. (eds.) *ICISC 22: 25th International Conference on Information Security and Cryptology*. Lecture Notes in Computer Science, vol. 13849, pp. 292–314. Springer, Cham (Nov / Dec 2022). [https://doi.org/10.1007/978-3-031-29371-9\\_15](https://doi.org/10.1007/978-3-031-29371-9_15)
9. Anastasova, M., Bisheh-Niasar, M., Azarderakhsh, R., Kermani, M.M.: Compressed SIKE round 3 on ARM Cortex-M4. *Cryptology ePrint Archive*, Report 2021/1511 (2021), <https://eprint.iacr.org/2021/1511>
10. Arm Limited: Arm Cortex-M4 datasheet (2020), <https://developer.arm.com/documentation/102832/latest/>
11. Arm Limited: Arm® Cortex®-M4 processor technical reference manual revision r0p1 (2020), <https://developer.arm.com/documentation/100166/0001/>
12. Arm Limited: Arm®v7-M architecture reference manual (Feb 2021), <https://developer.arm.com/documentation/ddi0403/ee/>
13. Arm Limited: Arm CPU security update: Timing side channel attacks on TrustZone enabled Cortex-M based systems (2025), <https://developer.arm.com/documentation/110390/1-1/>
14. Baldi, M., Barenghi, A., Beckwith, L., Biasse, J., Chou, T., Esser, A., Gaj, K., Karl, P., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.O., Santini, P., Wallace, R., Zwyedinger, F.: LESS — Linear Equivalence Signature Scheme. Tech. rep., National Institute of Standards and Technology (2024), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>



15. Becker, H., Hwang, V., Kannwischer, M.J., Panny, L., Yang, B.Y.: Efficient multiplication of somewhat small integers using number-theoretic transforms. In: Cheng, C.M., Akiyama, M. (eds.) IWSEC 22: 17th International Workshop on Security, Advances in Information and Computer Security. Lecture Notes in Computer Science, vol. 13504, pp. 3–23. Springer, Cham (Aug / Sep 2022). [https://doi.org/10.1007/978-3-031-15255-9\\_1](https://doi.org/10.1007/978-3-031-15255-9_1)
16. Bernstein, D.J.: Cache-timing attacks on AES (2005), <https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>
17. Bernstein, D.J.: djb sort software library. <https://sorting.cr.yp.to/> (2019)
18. Bernstein, D.J., Chuengsatiansup, C., Lange, T., van Vredendaal, C.: NTRU prime: Reducing attack surface at low cost. In: Adams, C., Camenisch, J. (eds.) SAC 2017: 24th Annual International Workshop on Selected Areas in Cryptography. Lecture Notes in Computer Science, vol. 10719, pp. 235–260. Springer, Cham (Aug 2017). [https://doi.org/10.1007/978-3-319-72565-9\\_12](https://doi.org/10.1007/978-3-319-72565-9_12)
19. Bettaieb, S., Bidoux, L., Budroni, A., Palumbi, M., Perin, L.P.: Enabling PERK and other MPC-in-the-head signatures on resource-constrained devices. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(4), 84–109 (2024). <https://doi.org/10.46586/tches.v2024.i4.84-109>
20. Biham, E.: A fast new DES implementation in software. In: Biham, E. (ed.) Fast Software Encryption, 4th International Workshop, FSE '97, Haifa, Israel, January 20–22, 1997, Proceedings. Lecture Notes in Computer Science, vol. 1267, pp. 260–272. Springer (1997). <https://doi.org/10.1007/BFB0052352>, <https://doi.org/10.1007/BFB0052352>
21. Budroni, A., Canales-Martínez, I.A., Perin, L.P.: SoK: Methods for sampling random permutations in post-quantum cryptography. Cryptology ePrint Archive, Report 2024/008 (2024), <https://eprint.iacr.org/2024/008>
22. Chen, C., Danba, O., Hoffstein, J., Hulsing, A., Rijneveld, J., Schanck, J.M., Schwabe, P., Whyte, W., Zhang, Z., Saito, T., Yamakawa, T., Xagawa, K.: NTRU. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
23. Chen, M.S., Chou, T.: Classic McEliece on the ARM Cortex-M4. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(3), 125–148 (2021). <https://doi.org/10.46586/tches.v2021.i3.125-148>, <https://tches.iacr.org/index.php/TCHES/article/view/8970>
24. Chung, C.M.M., Hwang, V., Kannwischer, M.J., Seiler, G., Shih, C.J., Yang, B.Y.: NTT multiplication for NTT-unfriendly rings. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(2), 159–188 (2021). <https://doi.org/10.46586/tches.v2021.i2.159-188>, <https://tches.iacr.org/index.php/TCHES/article/view/8791>
25. Dhem, J.F., Koeune, F., Leroux, P.A., Mestré, P., Quisquater, J.J., Willems, J.L.: A practical implementation of the timing attack. In: Quisquater, J.J., Schneier, B. (eds.) Smart Card Research and Applications. pp. 167–182. Springer Berlin Heidelberg, Berlin, Heidelberg (2000)
26. Durstenfeld, R.: Algorithm 235: Random permutation. Commun. ACM **7**(7), 420 (Jul 1964). <https://doi.org/10.1145/364520.364540>, <https://doi.org/10.1145/364520.364540>
27. Filho, D.L.G., Silva, T.S.R., López-Hernández, J.C.: Efficient isochronous fixed-weight sampling with applications to NTRU. IACR Communications in Cryptology (CiC) **1**(2), 14 (2024). <https://doi.org/10.62056/a6n59qgqxq>

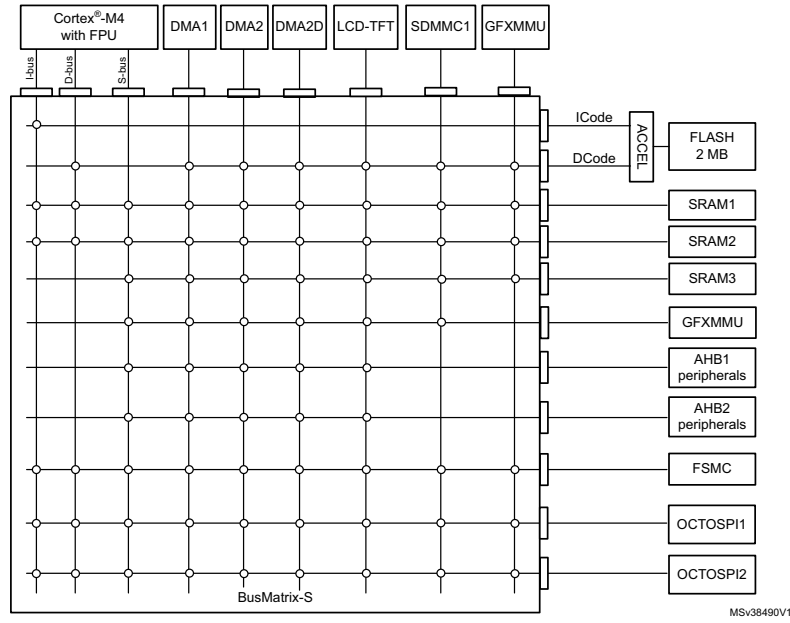
28. Fisher, R.A., Yates, F.: Statistical Tables for Biological, Agricultural and Medical Research. Oliver & Boyd, London, 6th edn. (1938)
29. Hamburg, M.: Accelerating AES with vector permute instructions. In: Clavier, C., Gaj, K. (eds.) Cryptographic Hardware and Embedded Systems – CHES 2009. Lecture Notes in Computer Science, vol. 5747, pp. 18–32. Springer, Berlin, Heidelberg (Sep 2009). [https://doi.org/10.1007/978-3-642-04138-9\\_2](https://doi.org/10.1007/978-3-642-04138-9_2)
30. Huang, J., Zhang, J., Zhao, H., Liu, Z., Cheung, R.C.C., Koç, Ç.K., Chen, D.: Improved plantard arithmetic for lattice-based cryptography. IACR Transactions on Cryptographic Hardware and Embedded Systems **2022**(4), 614–636 (2022). <https://doi.org/10.46586/tches.v2022.i4.614-636>
31. Kannwischer, M.J., Krausz, M., Petri, R., Yang, S.Y.: pqm4: Benchmarking NIST additional post-quantum signature schemes on microcontrollers. Cryptology ePrint Archive, Report 2024/112 (2024), <https://eprint.iacr.org/2024/112>
32. Kannwischer, M.J., Rijneveld, J., Schwabe, P., Stoffelen, K.: pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. Cryptology ePrint Archive, Report 2019/844 (2019), <https://eprint.iacr.org/2019/844>
33. Käsper, E., Schwabe, P.: Faster and timing-attack resistant AES-GCM. In: Clavier, C., Gaj, K. (eds.) Cryptographic Hardware and Embedded Systems – CHES 2009. Lecture Notes in Computer Science, vol. 5747, pp. 1–17. Springer, Berlin, Heidelberg (Sep 2009). [https://doi.org/10.1007/978-3-642-04138-9\\_1](https://doi.org/10.1007/978-3-642-04138-9_1)
34. Knuth, D.E.: The Art of Computer Programming, Volume 3: Sorting and Searching. Addison-Wesley, Boston, 2nd edn. (1997)
35. Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T., Schwarz, M., Yarom, Y.: Spectre attacks: Exploiting speculative execution. In: 2019 IEEE Symposium on Security and Privacy. pp. 1–19. IEEE Computer Society Press (May 2019). <https://doi.org/10.1109/SP.2019.00002>
36. Kocher, P.C.: Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In: Koblitz, N. (ed.) Advances in Cryptology – CRYPTO’96. Lecture Notes in Computer Science, vol. 1109, pp. 104–113. Springer, Berlin, Heidelberg (Aug 1996). [https://doi.org/10.1007/3-540-68697-5\\_9](https://doi.org/10.1007/3-540-68697-5_9)
37. Könighofer, R.: A fast and cache-timing resistant implementation of the AES. In: Malkin, T. (ed.) Topics in Cryptology – CT-RSA 2008. Lecture Notes in Computer Science, vol. 4964, pp. 187–202. Springer, Berlin, Heidelberg (Apr 2008). [https://doi.org/10.1007/978-3-540-79263-5\\_12](https://doi.org/10.1007/978-3-540-79263-5_12)
38. Lemire, D.: Fast random integer generation in an interval. ACM Trans. Model. Comput. Simul. **29**(1) (jan 2019). <https://doi.org/10.1145/3230636>, <https://doi.org/10.1145/3230636>
39. Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Fogh, A., Horn, J., Mangard, S., Kocher, P., Genkin, D., Yarom, Y., Hamburg, M.: Meltdown: Reading kernel memory from user space. In: Enck, W., Felt, A.P. (eds.) USENIX Security 2018: 27th USENIX Security Symposium. pp. 973–990. USENIX Association (Aug 2018)
40. Matsui, M.: How far can we go on the x64 processors? In: Robshaw, M.J.B. (ed.) Fast Software Encryption – FSE 2006. Lecture Notes in Computer Science, vol. 4047, pp. 341–358. Springer, Berlin, Heidelberg (Mar 2006). [https://doi.org/10.1007/11799313\\_22](https://doi.org/10.1007/11799313_22)
41. Matsui, M., Nakajima, J.: On the power of bitslice implementation on intel core2 processor. In: Paillier, P., Verbauwhede, I. (eds.) Cryptographic Hardware and Embedded Systems – CHES 2007. Lecture Notes in Computer Science, vol. 4727,

- pp. 121–134. Springer, Berlin, Heidelberg (Sep 2007). [https://doi.org/10.1007/978-3-540-74735-2\\_9](https://doi.org/10.1007/978-3-540-74735-2_9)
42. Microelectronics, S.: RM0432 reference manual: STM32L4+ series advanced Arm®-based 32-bit MCUs (Jun 2021), [https://www.st.com/resource/en/reference\\_manual/rm0432-stm32l4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf](https://www.st.com/resource/en/reference_manual/rm0432-stm32l4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf)
  43. Pike, R., Locanthi, B., Reiser, J.: Hardware/software trade-offs for bitmap graphics on the blit. *Software: Practice and Experience* **15**(2), 131–151 (1985). <https://doi.org/https://doi.org/10.1002/spe.4380150203>, <https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.4380150203>
  44. Pornin, T.: Why constant-time crypto? (2018), <https://bearssl.org/constanttime.html>
  45. Pornin, T.: Falcon on ARM cortex-M4: an update. *Cryptology ePrint Archive, Report 2025/123* (2025), <https://eprint.iacr.org/2025/123>
  46. Rodrigues, C., Oliveira, D., Pinto, S.: BUSTed!!! Microarchitectural side-channel attacks on the MCU bus interconnect. In: 2024 IEEE Symposium on Security and Privacy. pp. 3679–3696. IEEE Computer Society Press (May 2024). <https://doi.org/10.1109/SP54263.2024.00062>
  47. Schulman, A.: Unauthorized Windows 95: A Developer’s Guide to Exploring the Foundations of Windows “Chicago”. International Data Group Company (1994)
  48. Schwabe, P., Stoffelen, K.: All the AES you need on Cortex-M3 and M4. In: Avanzi, R., Heys, H.M. (eds.) SAC 2016: 23rd Annual International Workshop on Selected Areas in Cryptography. *Lecture Notes in Computer Science*, vol. 10532, pp. 180–194. Springer, Cham (Aug 2016). [https://doi.org/10.1007/978-3-319-69453-5\\_10](https://doi.org/10.1007/978-3-319-69453-5_10)
  49. Sendrier, N.: Secure sampling of constant-weight words – application to BIKE. *Cryptology ePrint Archive, Report 2021/1631* (2021), <https://eprint.iacr.org/2021/1631>
  50. Tsunoo, Y., Saito, T., Suzaki, T., Shigeri, M., Miyauchi, H.: Cryptanalysis of DES implemented on computers with cache. In: Walter, C.D., Koç, Çetin Kaya., Paar, C. (eds.) *Cryptographic Hardware and Embedded Systems – CHES 2003. Lecture Notes in Computer Science*, vol. 2779, pp. 62–76. Springer, Berlin, Heidelberg (Sep 2003). [https://doi.org/10.1007/978-3-540-45238-6\\_6](https://doi.org/10.1007/978-3-540-45238-6_6)
  51. Tsunoo, Y., Tsujihara, E., Minematsu, K., Miyauchi, H.: Cryptanalysis of block ciphers implemented on computers with cache. In: *Proceedings of the International Symposium on Information Theory and Its Applications, ISITA 2002*. pp. 803–806 (2002)
  52. Wang, Y., Paccagnella, R., He, E.T., Shacham, H., Fletcher, C.W., Kohlbrenner, D.: Hertzbleed: Turning power side-channel attacks into remote timing attacks on x86. In: Butler, K.R.B., Thomas, K. (eds.) *USENIX Security 2022: 31st USENIX Security Symposium*. pp. 679–697. USENIX Association (Aug 2022)

## A Speeding up code execution from SRAM

The default (power-on reset) configuration of the NUCLEO-L4R5ZI development board, used for benchmarking in Section 5, unnecessarily slows down code execution from SRAM. This is usually not a concern, as embedded applications almost always executes from Flash memory, but it is critical for our code generation approach. We outline some steps to be taken to ensure that code executes from SRAM with minimal slowdowns.

In the STM32L4R5ZI microcontroller unit (MCU) at the core of the NUCLEO-L4R5ZI development board, the connection among the Cortex-M4 core, memory-mapped peripherals and different types of memory (Flash, SRAM, external, and memory-mapped peripherals) is realized by means of a bus matrix, a typical organization for MCUs from ST and other vendors. The particular layout of the bus matrix for the STM32L4R5ZI is displayed in Figure 2, which also highlights a characteristic of this particular ST part: the use of a banked memory organization, splitting SRAM into three banks (SRAM1, SRAM2 and SRAM3).



**Fig. 2.** STM32L4R5ZI memory bus architecture. Reproduced from [42, Figure 1].

Additionally, the memory map of the Cortex-M4 core is subdivided in 8 regions of 512 MB each (see [11, Figure 3-1 and Table 3-3] and [42, Figure 3]). We are mostly concerned with the Code and SRAM regions, spanning the memory address ranges of  $0x0000\ 0000\text{--}0x1FFF\ FFFF$  and  $0x2000\ 0000\text{--}0x3FFF\ FFFF$ , respectively.

It is noted that the Cortex-M4 core is connected to the bus matrix through three different bus interfaces (see [42, Sections 2.1.1–2.1.3] and [11, Section 2.2.1]):

- I-bus/I-Code memory interface: used for instruction fetches from the Code region.
- D-bus/D-Code memory interface: used for data (literal loads) and debug access from the Code region.

- S-bus/System interface: used for instruction fetches, data and debug access from other regions, in particular the SRAM region.

Regarding the System interface, ARM’s Cortex-M4 Technical Reference Manual [11, p. 2-25] states that:

#### **Pipelined instruction fetches**

To provide a clean timing interface on the system bus, instruction and vector fetch requests to this bus are registered.

This results in an extra cycle of latency because instructions fetched from the system bus take two cycles. This also means that back-to-back instruction fetches from the system bus are not possible.

**Note:** Instruction fetch requests to the ICode bus are not registered. Performance critical code must run from the ICode interface.

Therefore, it is important to ensure that generated code is run from the ICode bus to attain maximum performance. However, one runs into the roadblock that the SRAM banks are accessible from the SRAM region, starting at address 0x2000 0000, rather than the Code region that starts at address 0x0000 0000. However, note in Figure 2 that SRAM1 and SRAM2 (but not SRAM3) have connections to the I-bus and D-bus, in addition to the S-bus.

Although this indicates that access via the I-bus and D-bus are possible, an additional configuration step is required to remap the SRAM region into the Code region, using the `SYSCFG_MEMRMP` register of the STM32L4R5ZI [42, Section 9.2.1]. When this configuration is performed, any accesses to addresses starting at 0x0000 0000 are remapped to the same address with the 29th bit set, i.e. starting from 0x2000 0000; note that this remapping is more precisely an alias, since accesses directly to the SRAM region, to addresses starting at 0x2000 0000, are still possible. This procedure must be performed carefully, due to the interrupt vector table that is normally stored in Flash memory at 0x0800 0000 and remapped (per the default configuration) to address 0x0000 0000. When the remap is performed, the vector table must be available at address 0x2000 0000, which must be reserved by editing the linker script, and a copy made from Flash to this address in SRAM1, prior to performing the remap. It is also necessary to enable the clock for the `SYSCFG` peripheral, and interrupts must be disabled during the remapping to avoid race conditions; the exact procedure can be reviewed in our source code package, in the `remap_setup` function found in either `hw/ha1.c` file of the microbenchmarking project or `common/ha1-opencm3.c` file of the `pqm4` project.

Recall from Figure 2 that only SRAM1 and SRAM2 banks are connected to the I-bus. Therefore, another important point is to ensure that arrays storing generated code must be declared in either one of these banks, rather than SRAM3. The exact placement of variables is determined by the linker script; we will refrain from discussing all intricacies of linker script writing, and instead point out that for a typical linker script organization, global variables are placed starting at the beginning of SRAM (i.e. SRAM1 in our case), whereas the stack

pointer is set to the end of SRAM (i.e. SRAM3) and grows down. Therefore, code arrays should be declared as global variables to place them in SRAM1, and this is the strategy followed in our implementations.

Unfortunately, this is still not sufficient to ensure the code runs from the Code region. Linker scripts place references to any SRAM object at addresses starting at 0x2000 0000; this must not be changed directly in the linker script, as the MCU boots up with the default remapping configuration, aliasing Flash addresses starting at 0x0800 0000 to 0x0000 0000. Therefore, prior to the remapping, SRAM addresses must remain in their original range starting at 0x2000 0000. Also, accesses to the code array during code generation can use the default address in the SRAM region; the issue raised above pertains only to execution of the code. Thus, only the function pointer, used to call the generated code, actually needs to point to an address in the Code range, i.e. with the 29th bit cleared. We solve this conundrum by casting the code array to a function pointer while clearing the 29th bit. Concretely, we illustrate the required change for lines 7 and 8 of the function `array32x8_read` in Listing 1.1; similar changes can be performed for any implementations using the code generation approach:

```
uint32_t (*f)(uint32_t *) =
 (uint32_t (*)(uint32_t *))((uint32_t)code - 0x20000000 + 1);
```

In order to quantify the effect of such configuration changes, we display in Table 7 microbenchmarks for permutation ( $n \in \{79, 83, 112, 116\}$ ) and fixed-weight ( $n = 508$ ) sampling, using the code generation approach, for both the default configuration and an optimized one. The same methodology as Section 5.1 is followed, and therefore cycle counts for the optimized configuration are identical to those of Tables 2 and 3. It is seen that the speedups from using an optimized configuration range from 3.3–5.7%.

**Table 7.** Microbenchmarks for permutation and fixed-weight sampling, using either a default or optimized configuration. The parameter  $n$  is the length of the permutations/vectors. Speedup is reported as the ratio between cycle counts of the default and optimized configuration. All cycle counts are exclusive of pseudorandomness sampling.

| Configuration | $n = 79$    | $n = 83$    | $n = 112$   | $n = 116$   | $n = 508$    |
|---------------|-------------|-------------|-------------|-------------|--------------|
| Optimized     | <b>4911</b> | <b>5225</b> | <b>7564</b> | <b>7883</b> | <b>44003</b> |
| Default       | 5184        | 5519        | 7993        | 8328        | 45461        |
| Speedup       | 1.06×       | 1.06×       | 1.06×       | 1.06×       | 1.03×        |